

Inteligência Artificial



Agentes baseado em Objetivos

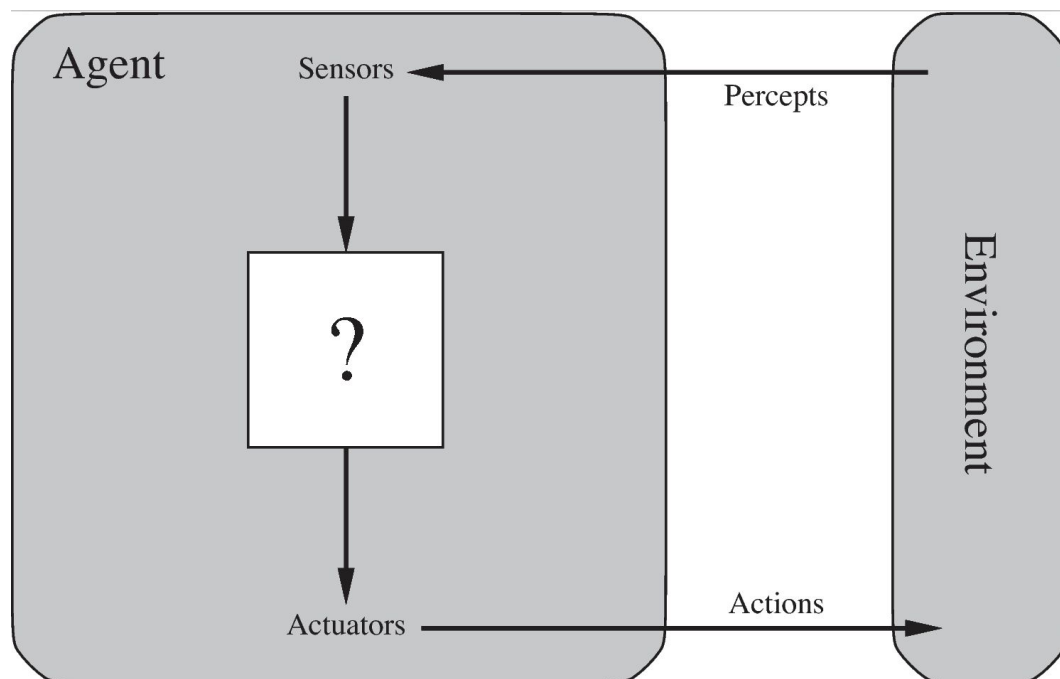
MATA64

Ricardo Araújo Rios

Lab.IA
Instituto de Computação - UFBA

1

Agentes Inteligentes



2

Arquitetura de Agentes inteligentes



- Agente reativo simples
- Agente reativo baseado em modelo
- Agente reativo baseado em objetivo
- Agente reativo baseado em utilidade
- Agente com aprendizagem

Agente reativo simples

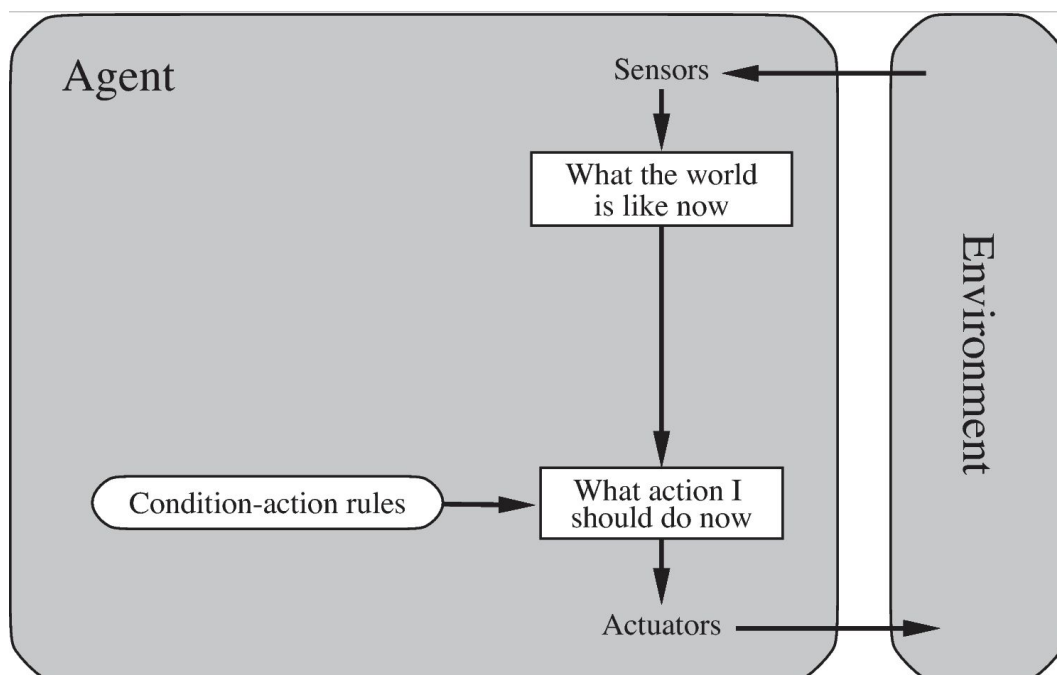
- Tipo mais simples de agente
- Selecionam ações com base na percepção atual e ignora o histórico de percepções
- O funcionamento do agente reativo é baseado em regras de condição-ação:
 - **If** condição **then** ação



Agente reativo simples

- Vantagens:
 - Regras condição-ação: representação inteligível, modular e eficiente
 - **Se** velocidade > 60 **então** multar
 - São utilizados em ambientes mais complexos
 - **Se** motorista da frente está pisando freio **então** pisar no freio
- Desvantagens
 - Inteligência limitada
 - A agente funcionará apenas se a decisão correta puder ser tomada com base na percepção atual (ambiente observável)

Agente reativo simples



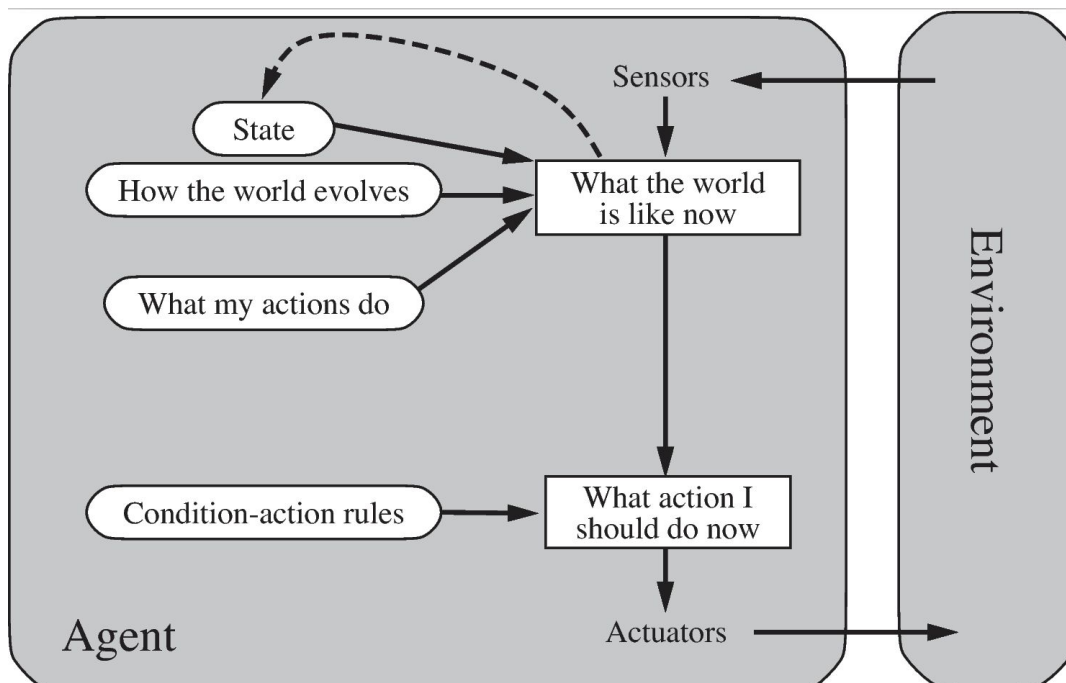


Agente Reativo Baseado em Modelo

- Em ambiente parcialmente observável, o agente precisa monitorar a parte do mundo que ele não pode ver agora.
- Agente mantém um estado interno com histórico de percepções
- Dois tipos de conhecimento implementados no agente
 - Como o mundo evolui independente do agente
 - Como o mundo reage a partir de uma ação do agente
- Modelo do mundo: como o mundo funciona

7

Agente Reativo Baseado em Modelo



8

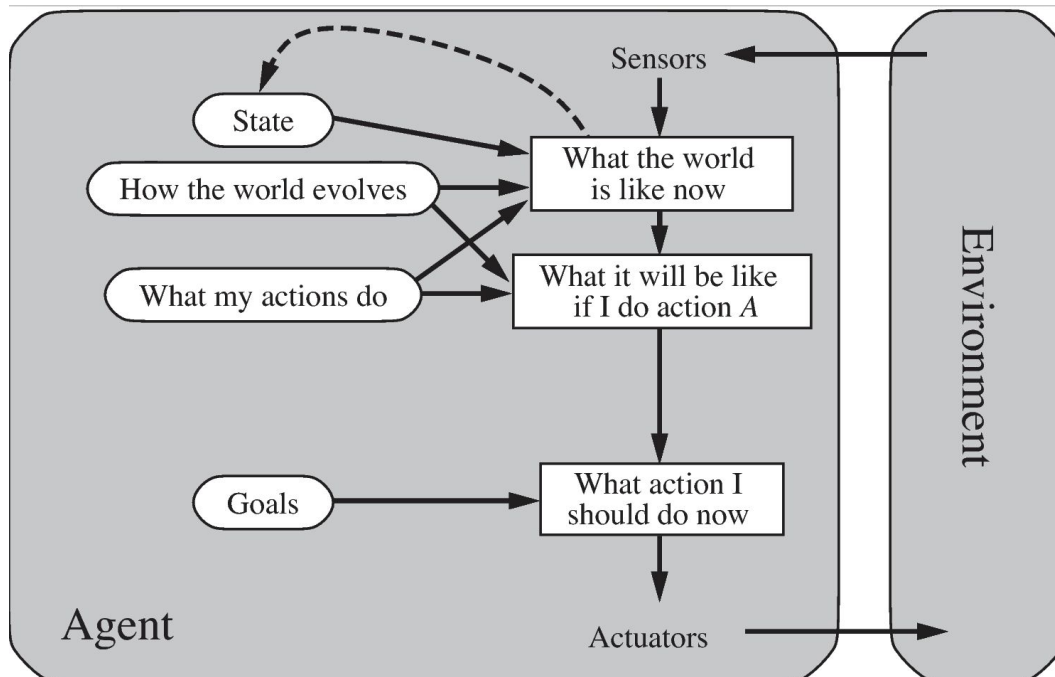
Agente Reativo Baseado em Modelo

- Raramente é possível determinar exatamente o estado atual em ambientes parcialmente observáveis
- **Como o mundo é agora** é o melhor palpite do agente
- A incerteza sobre o estado atual pode ser inevitável, mas o agente precisará tomar uma decisão

Agente reativo baseado em objetivo

- Conhecer o estado do ambiente é suficiente para decidir o que fazer?
 - Táxi pode seguir por 3 estradas. O que fazer?
 - Qual o objetivo?
 - “O que acontece se eu fizer isso?”
- Programa agente pode combinar objetivos que descrevem situações desejáveis com o modelo.
- Seleção baseada em objetivos pode ser:
 - Direta
 - Complexa: busca e planejamento

Agente reativo baseado em objetivo



11

Agente reativo baseado na utilidade

- Objetivos são suficientes para gerar um comportamento de alta qualidade?
 - Objetivos: “feliz” ou “infeliz”
 - E sobre o grau de felicidade?
 - Ex.: Chego ao destino (**feliz**) com baixo consumo de combustível e rapidez (**muito feliz**).
- Felicidade = Utilidade

12

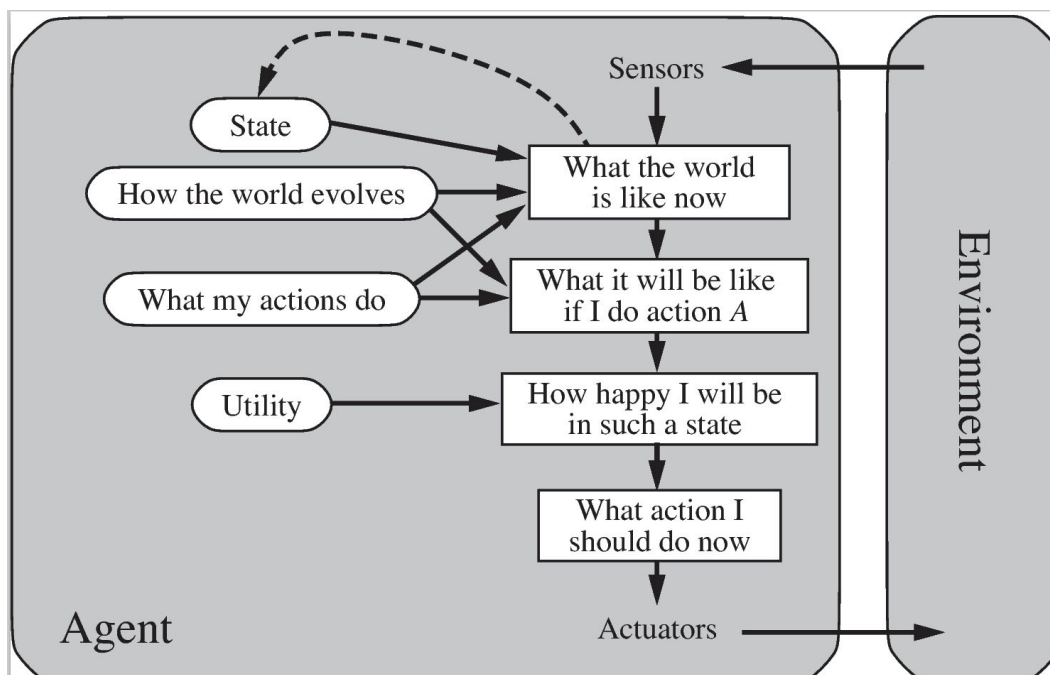


Agente reativo baseado na utilidade

- Função de utilidade é a internalização da medida de desempenho
- Se a função de utilidade interna e a medida externa de desempenho estiverem de acordo, um agente que escolhe ações que maximizem sua utilidade será racional de acordo com a medida de desempenho externa.

13

Agente reativo baseado na utilidade



14

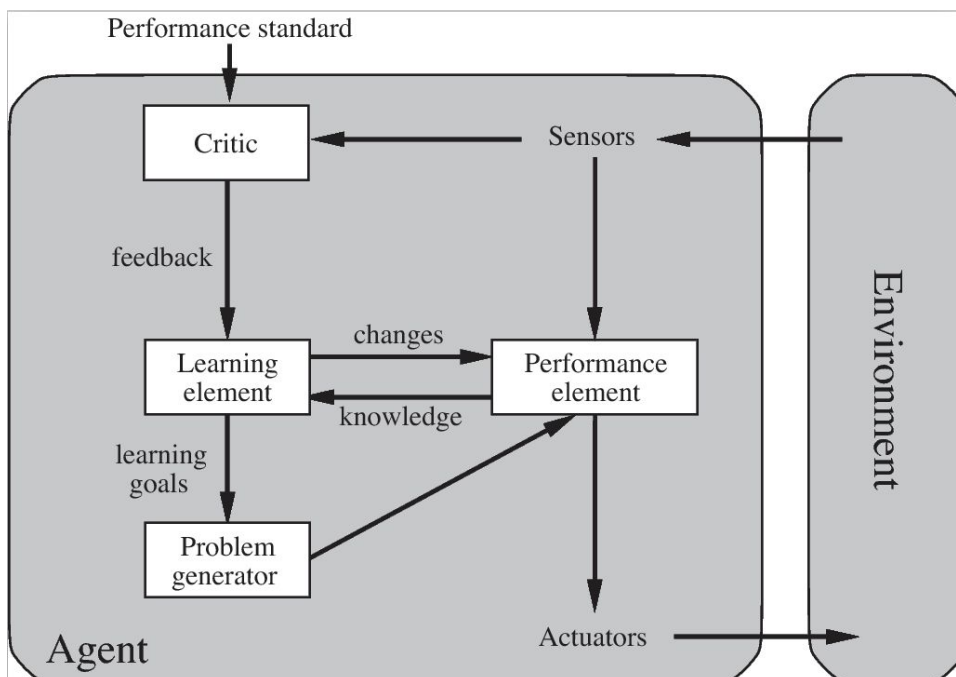


Agente reativo com aprendizagem

- Para os agentes descritos anteriormente, descrevemos programas para seleccionar ações
- Agentes podem aprender com base no ambiente e suas percepções?
- Vantagens
 - Podem executar em ambientes desconhecidos
 - Podem se tornar mais inteligente à medida que executam

15

Agente reativo com aprendizagem



16



Agente reativo com aprendizagem

- **Crítico:** observa como o agente está executando e determina o que precisa ser feito para executar melhor no futuro
- **Elemento de aprendizado:** responsável pelo aperfeiçoamento do das ações do agente
- **Elemento de desempenho:** seleção de ações externas
- **Gerador de problema:** generalização do que foi aprendido e exploração de situações não vistas → mínimo local para mínimo global



Agentes de Busca





Introdução

- Agentes Reativos
 - Modelo mais simples
 - Ações dependem de mapeamento direto dos estados
 - Dificuldade:
 - Mapeamento complexo para muitos estados
 - Alto custo de armazenamento e processamento



Introdução

- Agentes baseados em objetivo
 - Agentes de resolução de problemas
 - Não há mapeamento para tabela de estados
 - Maximiza a medida de desempenho definindo um objetivo
 - Objetivos limitam o que o agente precisa buscar e ações que precisam ser realizadas



Introdução

- **Formulação de Objetivos:**
 - Definição de um objetivo bem definido
 - Organização do comportamento
 - Conhecimento da situação atual do agente



Introdução

- **Formulação de Problemas:** processo de decidir quais ações e estados devem ser considerados para alcançar um objetivo
- **Importante:**
 - Evitar estados com níveis de detalhes incertos
 - Descobrir como agir: agora e futuro



Exemplo

- **Problema:** Uma pessoa precisa sair de uma cidade **A** e chegar o mais rápido possível a outra cidade **B**. Não existe um caminho direto ligando **A** e **B**.
- **Sem mapa (informações)**
 - Decisões aleatórias



Exemplo

- **Problema:** Uma pessoa precisa sair de uma cidade **A** e chegar o mais rápido possível a outra cidade **B**. Não existe um caminho direto ligando **A** e **B**.
- **Com mapa (informações)**
 - Informações sobre os estados que pode visitar
 - Ações baseadas no estado corrente e na informação existente
 - Conhecimento das rotas possíveis e escolha do melhor caminho



Exemplo

- **Problema:** Uma pessoa precisa sair de uma cidade **A** e chegar o mais rápido possível a outra cidade **B**. Não existe um caminho direto ligando **A** e **B**.
- **Descrição do Ambiente:**
 - Discreto: Conjunto de cidades ligando A e B é finito
 - Observável: Existe um mapa das estradas que ligam as cidades
 - Conhecido: Agente conhece os resultados de cada ação
 - Determinístico: Cada ação tem um resultado

25

Exemplo

- **Problema:** Uma pessoa precisa sair de uma cidade **A** e chegar o mais rápido possível a outra cidade **B**. Não existe um caminho direto ligando **A** e **B**.
 - **(P)** Viagem mais rápida
 - **(E)** Estradas que ligam cidades
 - **(A)** Braços e pernas (para dirigir um carro entre cidades)
 - **(S)** Olhos (para ver o mapa e onde está)



26



Introdução

- A solução para qualquer problema é uma sequência finita de ações.
- O processo de procurar pela sequência de ações que alcançam o objetivo é chamado de **busca**.



Definição de problemas

- Um problema pode ser definido formalmente por 5 componentes:
 - Estado Inicial
 - Ações
 - Modelo de Transição
 - Teste de Objetivo
 - Custo de Caminho





Definição de problemas

- **Estado Inicial**
 - Estado em que o agente começa
- **Ações**
 - Possíveis ações para o agente em um dado estado
- **Modelo de transição**
 - Descrição do que cada ação faz → resultado de uma ação a um estado
- **Teste de objetivo**
 - Determina se um estado é objetivo (condição de parada)
- **Custo de caminho**
 - Reflete a medida de desempenho

Definição de problemas

- **Estado Inicial**
 - Estado em que o agente começa
- **Ações**
 - Possíveis ações para o agente em um dado estado
- **Modelo de transição**
 - Descrição do que cada ação faz → resultado de uma ação a um estado
- **Teste de objetivo**
 - Determina se um estado é objetivo (condição de parada)
- **Custo de caminho**
 - Reflete a medida de desempenho

Solução ótima: menor custo de caminho entre todas as soluções





Problemas e soluções bem definidos

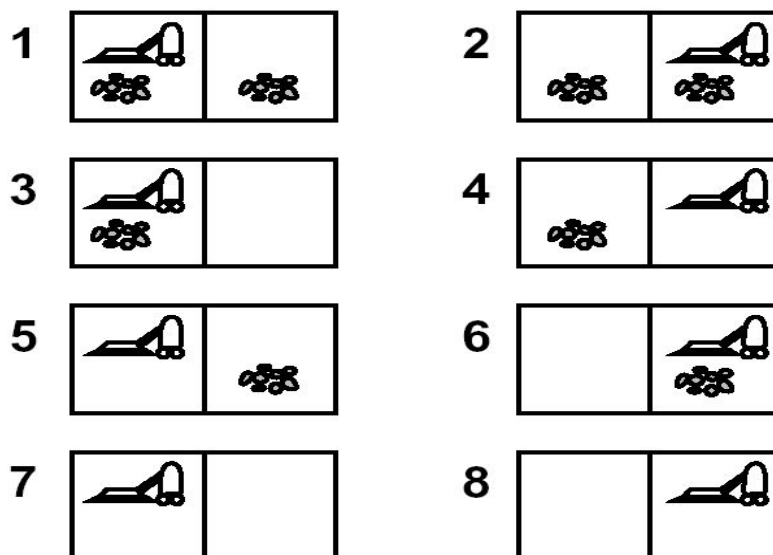
- Abstração na formulação de um problema
 - Ex: Ir de uma cidade para a outra
 - Vista da janela
 - Condições da estrada
 - Pousadas próximas
- Abstração nas ações
 - Somente a localização muda
 - Ex: Ir de uma cidade para a outra
 - Mudança de motorista
 - Olhar do motorista pela janela

Exemplos de problemas

- *Toy-problems*
 - Utilizados para ilustrar vários métodos de solução de problemas
 - Descrição exata
 - Utilizados por diferentes pesquisadores para comparar o desempenho de algoritmos
- *Real-world problems*
 - Difícil descrição
 - Formulação geral

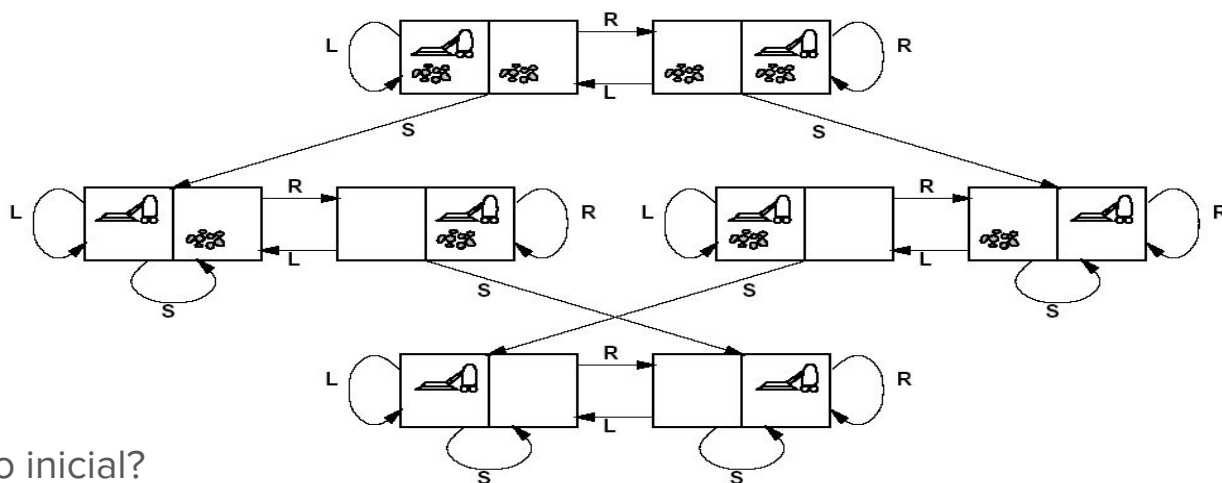


Exemplo: Mundo do Aspirador



33

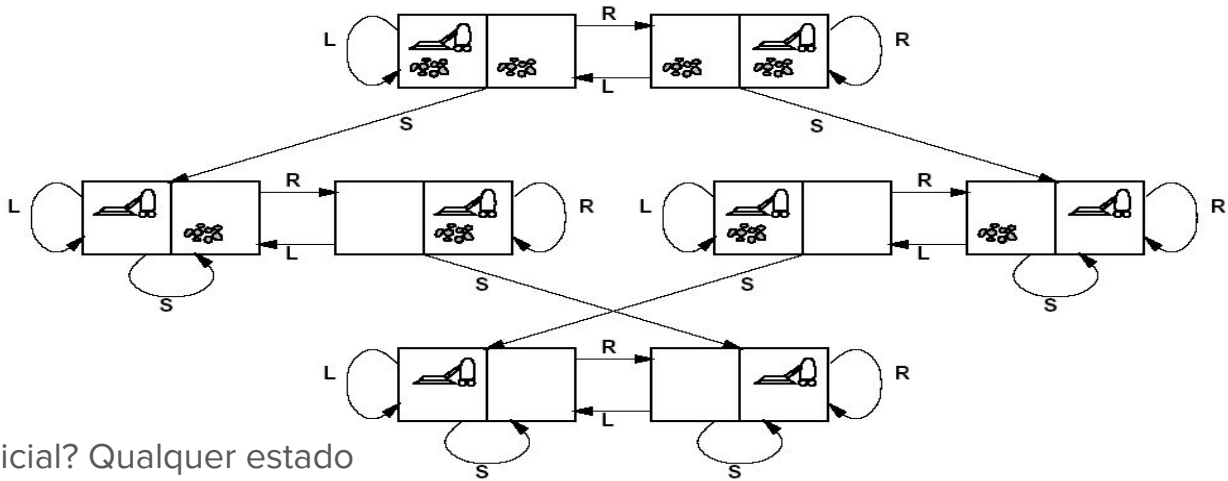
Exemplo: Mundo do Aspirador



- Estado inicial?
- Ações?
- Modelo de transição?
- Teste de objetivo?
- Custo de caminho?

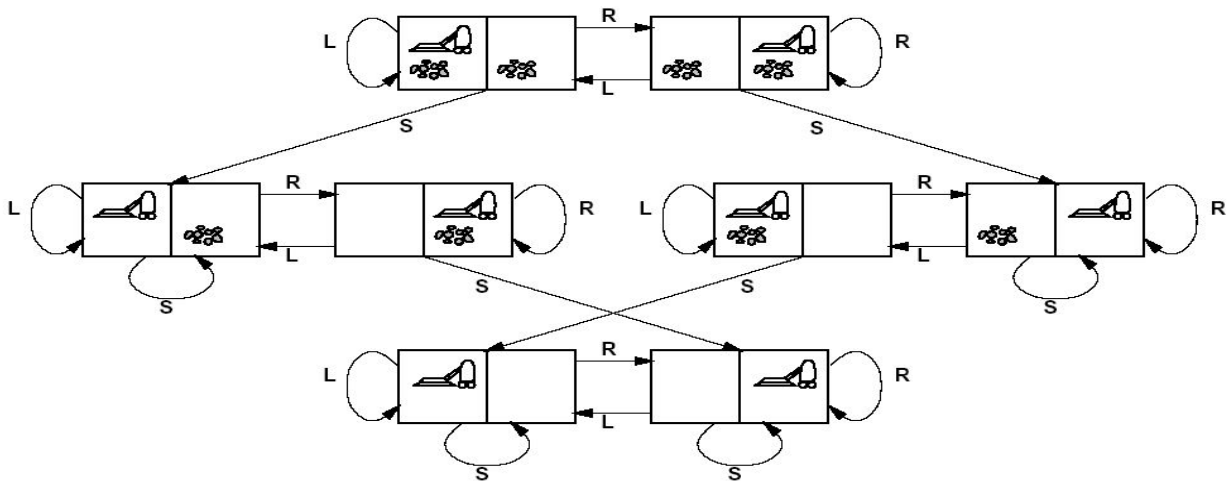
34

Exemplo: Mundo do Aspirador



- Estado inicial? Qualquer estado
- Ações? esquerda, direita, sucção
- Modelo de transição? Ações tem seus efeitos esperados, a não ser mover para a esquerda se já estiver na esquerda, mover para a direita se já estiver na direita e aspirar se o quadrado estiver limpo.

Exemplo: Mundo do Aspirador



- Teste de objetivo? Verifica se todos os quadrados estão limpos.
- Custo de caminho? Cada passo custa 1.



Exemplo: Quebra-Cabeça-8

5	4	
6	1	8
7	3	2

Estado Inicial

1	2	3
8		4
7	6	5

Estado Final

- Estado inicial?
- Ações?
- Modelo de transição?
- Teste de objetivo?
- Custo de caminho?

Exemplo: Quebra-Cabeça-8

5	4	
6	1	8
7	3	2

Estado Inicial

1	2	3
8		4
7	6	5

Estado Final

- Estado inicial? Qualquer estado
- Ações? Mover vazio para esquerda, para direita, para cima, para baixo
- Modelo de transição? Dado um estado e ação, ele devolve o estado resultante.
 - Ex: Se aplicarmos pra baixo para o estado inicial, o estado resultante terá comutado o 8 e o branco



Exemplo: Quebra-Cabeça-8

5	4	
6	1	8
7	3	2

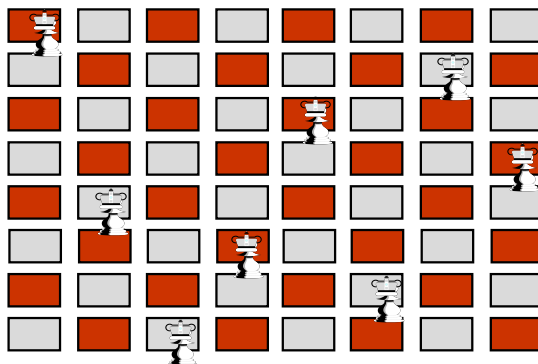
Estado Inicial

1	2	3
8		4
7	6	5

Estado Final

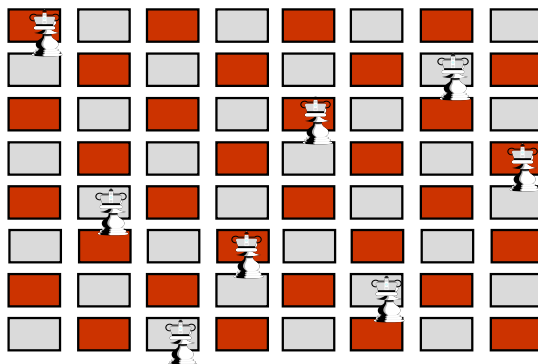
- Teste de objetivo? Verifica se chegou no estado fornecido como estado final.
- Custo de caminho? Cada passo custa 1.

Exemplo: 8 Rainhas



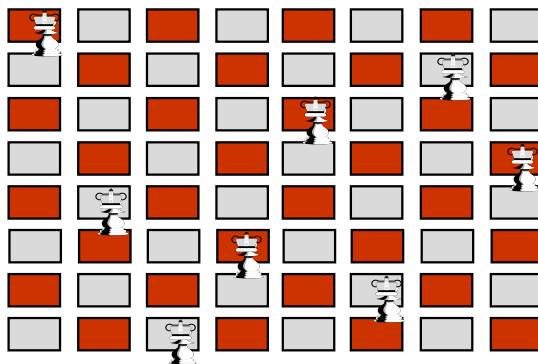
- Estado inicial?
- Ações?
- Modelo de transição?
- Teste de objetivo?
- Custo de caminho?

Exemplo: 8 Rainhas



- Estado inicial? nenhuma rainha no tabuleiro
- Ações? colocar uma rainha em qualquer quadrado vazio
- Modelo de transição? Devolver uma rainha adicionada em qualquer quadrado específico no tabuleiro.

Exemplo: 8 Rainhas



- Teste de objetivo? 8 rainhas no tabuleiro, sem ataque
- Custo de caminho? zero (apenas o estado final é interessante)

Problemas do mundo real

- Problema de roteamento
 - encontrar a melhor rota de um ponto a outro (aplicações: redes de computadores, planejamento militar, planejamento de viagens aéreas)
- Problemas de tour
 - visitar cada ponto pelo menos uma vez
- Caixeiro viajante
 - visitar cada cidade exatamente uma vez
 - encontrar o caminho mais curto
- Navegação de robôs
- Pesquisas na web



Exercício: Missionários e Canibais

- Três canibais e três missionários estão viajando juntos e chegam à margem de um rio. Eles desejam atravessar para a outra margem para, desta forma, continuar a viagem. O único meio de transporte disponível é um barco que comporta no máximo duas pessoas. Há uma outra dificuldade: em nenhum momento o número de canibais pode ser superior ao número de missionários pois desta forma os missionários estariam em grande perigo de vida. Como administrar a travessia?
- Quais os seus estados, estado inicial, ações, modelo de transição, teste de objetivo e custo do caminho?

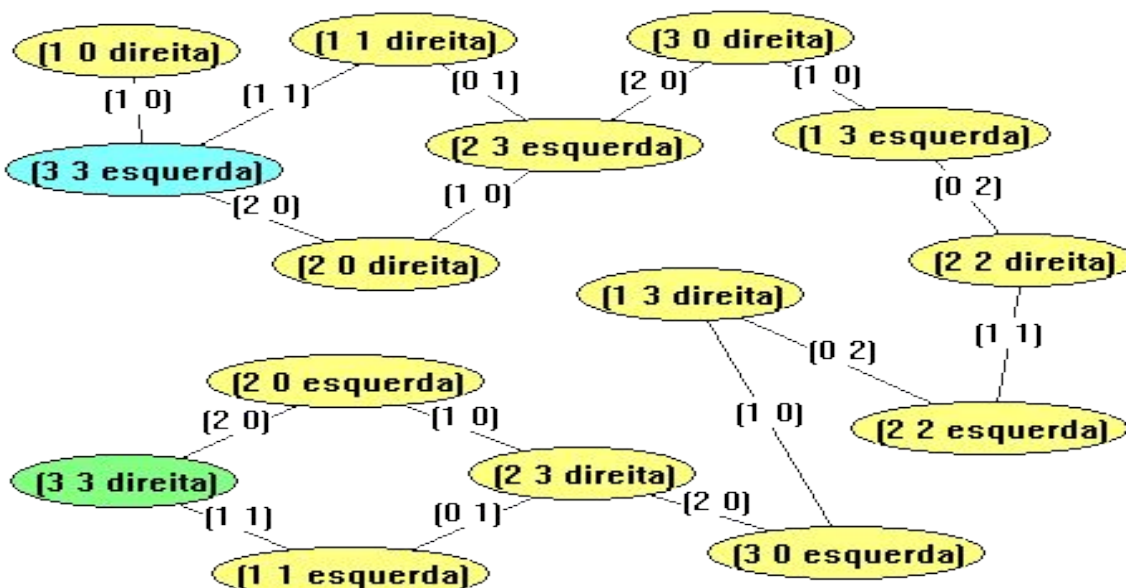


Exercício - Resposta

- Estados
 - um estado é uma sequência ordenada de dois números (# de canibais e missionários) e o local do bote no rio (inicialmente na margem esquerda)
 - Assim, o estado inicial é [3,3,esquerda]
- Ações
 - esquerda-direita, direita-esquerda
- Modelo de transição
 - a partir de cada estado, os operadores possíveis são: tomar um canibal e um missionário, dois missionários, dois canibais, um canibal ou um missionário, atravessando para a margem oposta
- Estado Final: [3,3,direita] (único)
- Custo do caminho: número de travessias



Missionários e Canibais





Busca de Soluções

- Uma solução é uma sequência de ações;
- Todas as sequências de ações possíveis começam a partir do estado inicial formando uma árvore de busca;
 - Nós: estados do problema
 - Arestas: ações
- Algoritmos de busca percorrem a árvore expandindo cada estado atual.
- Em seguida, explora uma opção deixando as demais (não exploradas) para próximas etapas.

47



Busca de Soluções

- O conjunto de nós folhas disponíveis para expansão é chamado de borda;
- Medidas de desempenho:
 - Completude
 - O algoritmo garante encontrar uma solução se existir?
 - Otimização
 - É uma solução ótima?
 - Complexidade de tempo
 - Tempo gasto para encontrar uma solução
 - Complexidade de espaço
 - Uso de memória para executar a busca

48



Busca de Soluções

- Teoricamente, em computação uma medida típica é o tamanho do grafo de espaço de estados: $|V| + |E|$
- Em IA, grafos são representados implicitamente pelo estado inicial, pelas ações e pelo modelo de transições.
- Complexidade em IA:
 - **b**: fator de ramificação número máximo de sucessores de cada nó
 - **d**: profundidade do nó objetivo menos profundo
 - **m**: comprimento máximo de qualquer caminho no espaço de estados
- **Tempo**: número de nós gerados
- **Espaço**: número máximo de nós armazenados em memória



Busca de Soluções

- Para avaliar a efetividade de um algoritmo de busca:
 - **(A)** Custo de busca: Número de nós gerados na busca
 - **(B)** Custo de solução: comprimento do caminho
 - **Custo Total = (A) + (B)**

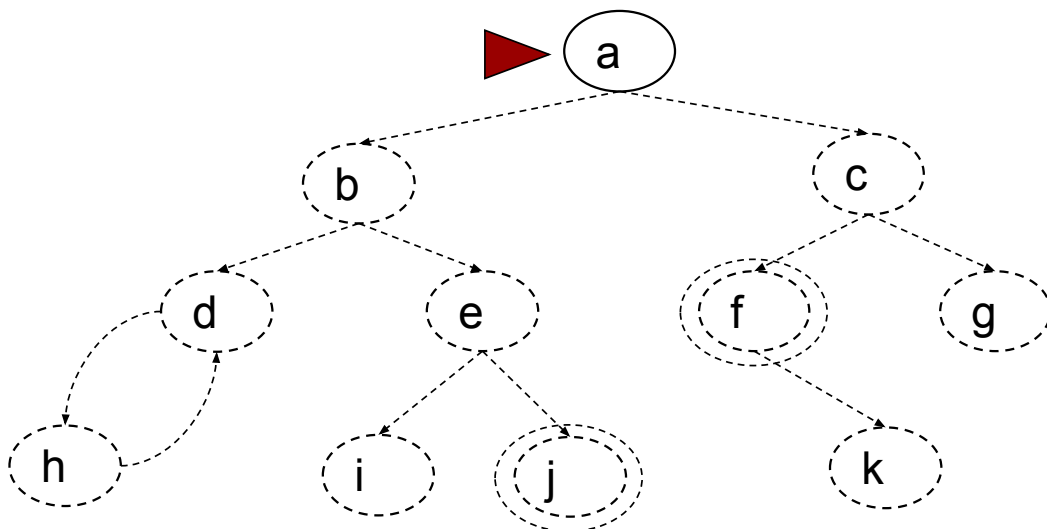


Busca em Largura

- (Breadth-First Search - BFS)
 - Estratégia simples: FIFO
 - Um nó é expandido e na sequência os filhos são avaliados
 - Nós de uma dada profundidade são expandidos antes dos nós dos níveis seguintes
 - É uma estratégia completa: nó objetivo numa profundidade d, será encontrado depois de avaliar todos os nós mais rasos.

51

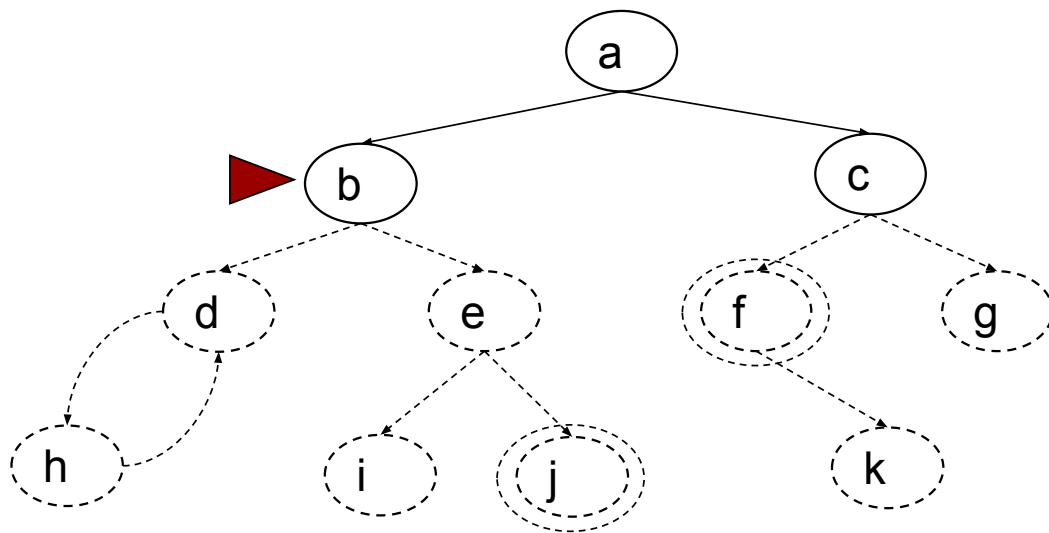
Busca em Largura



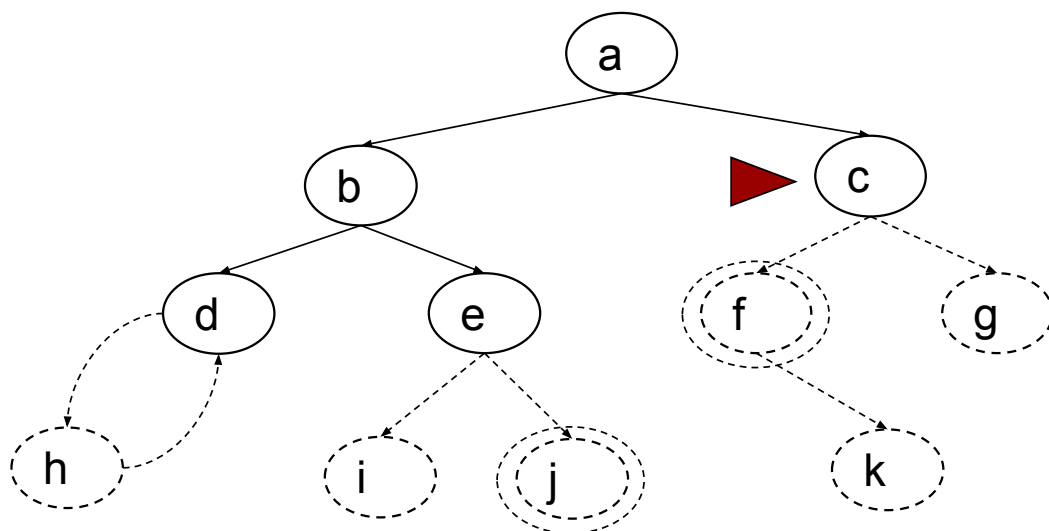
52



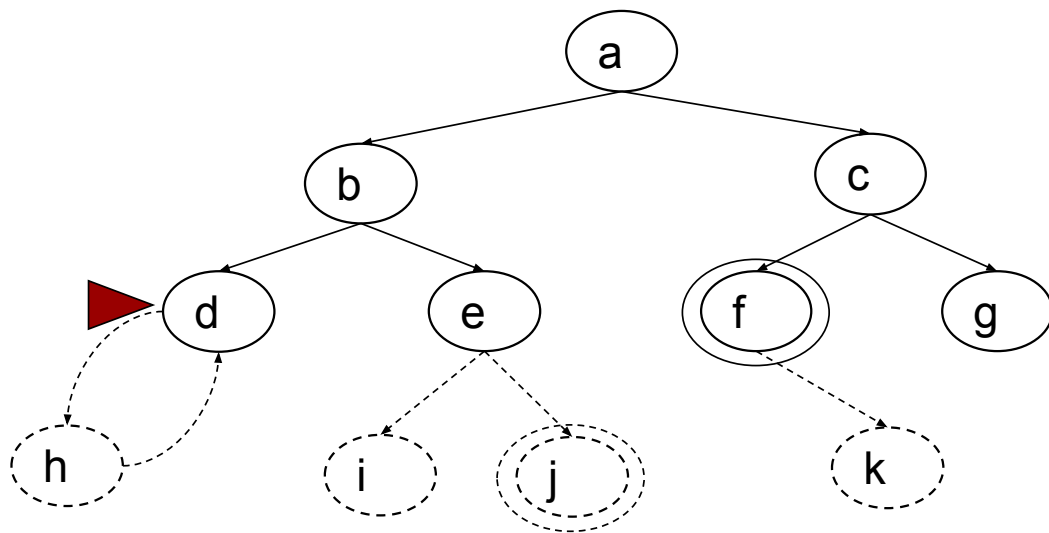
Busca em Largura



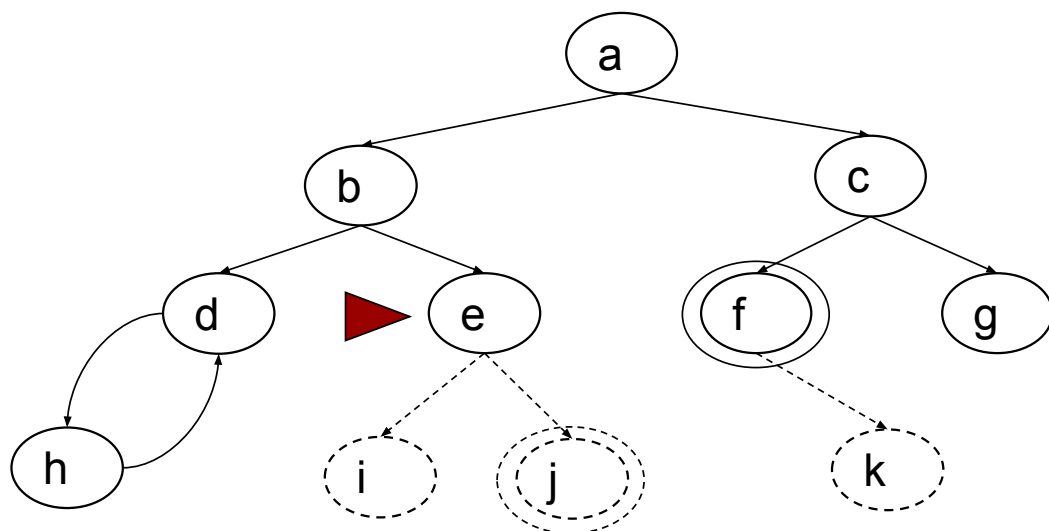
Busca em Largura



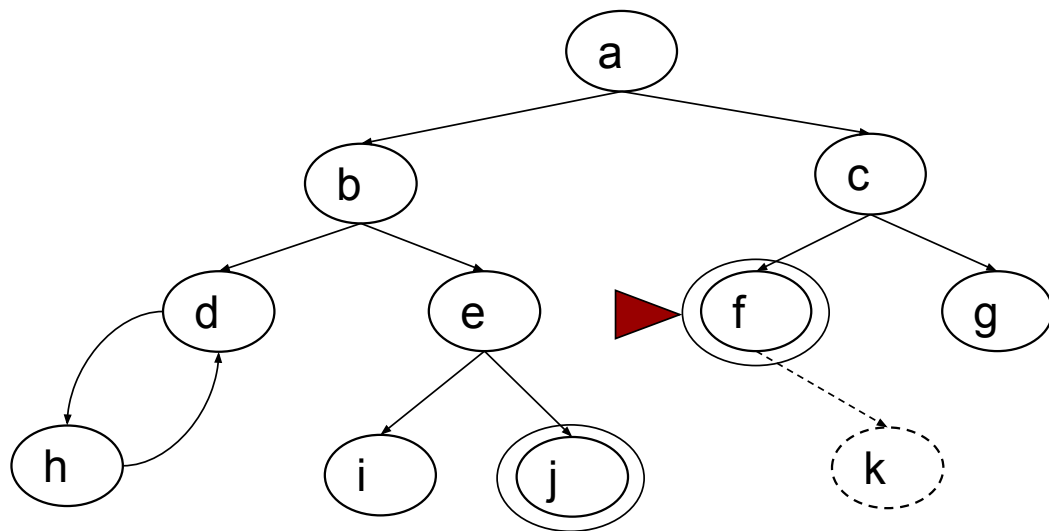
Busca em Largura



Busca em Largura

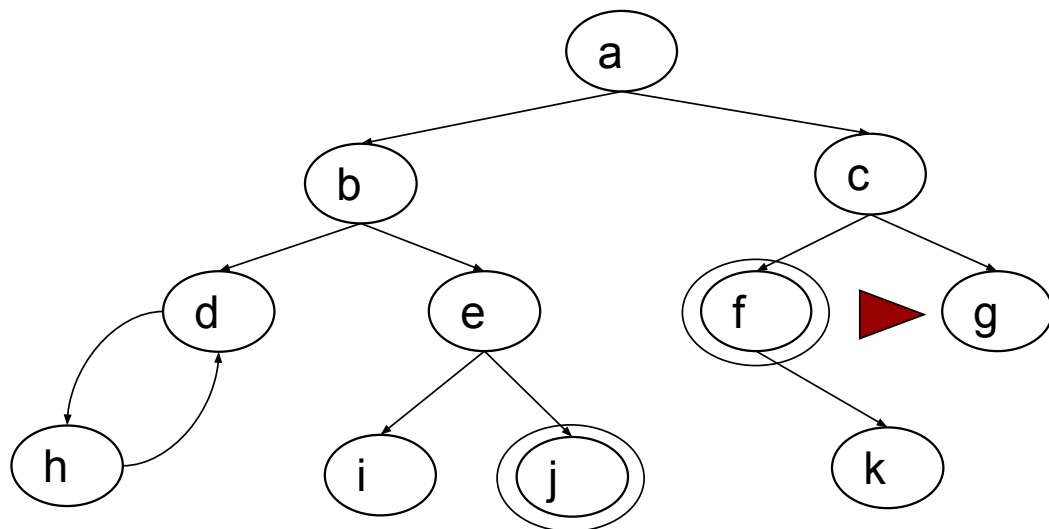


Busca em Largura



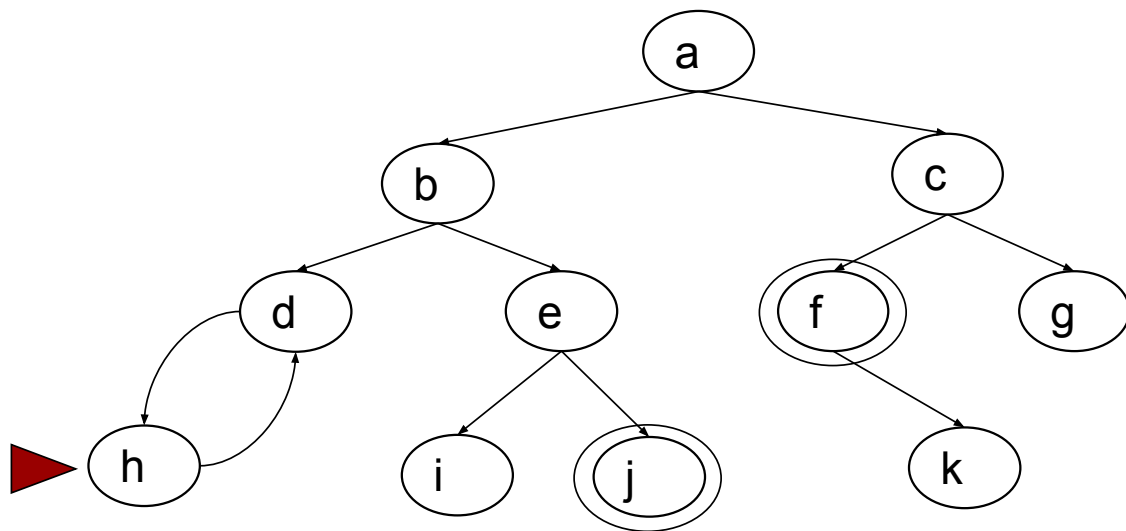
57

Busca em Largura

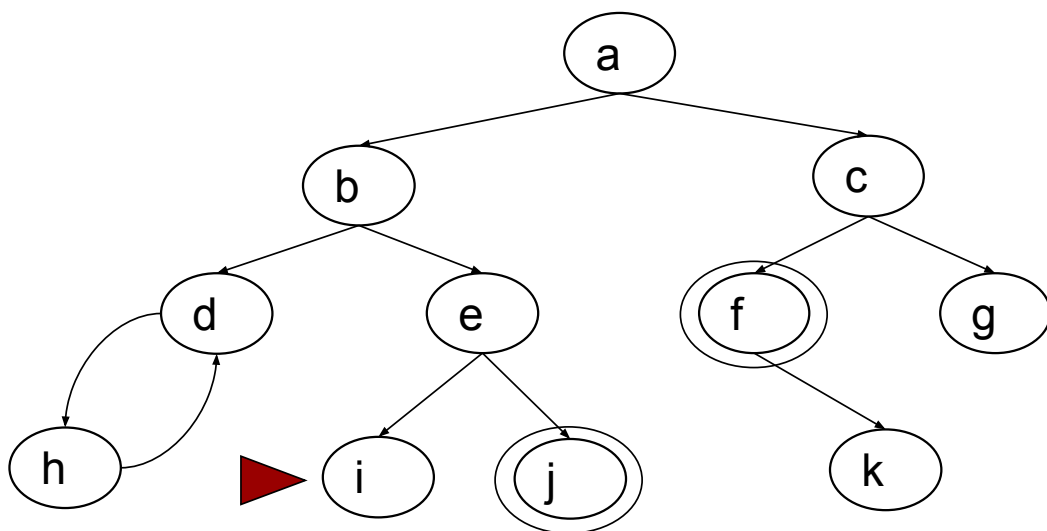


58

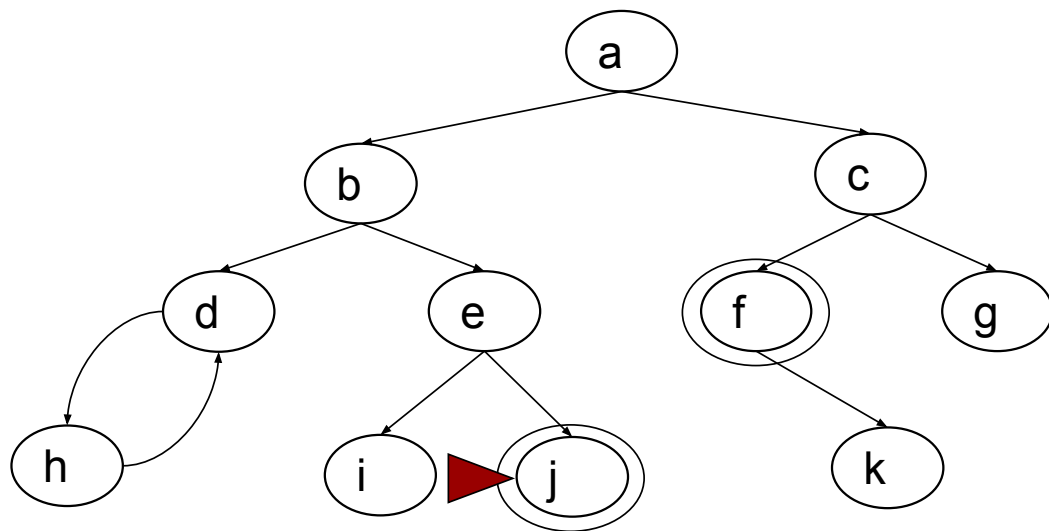
Busca em Largura



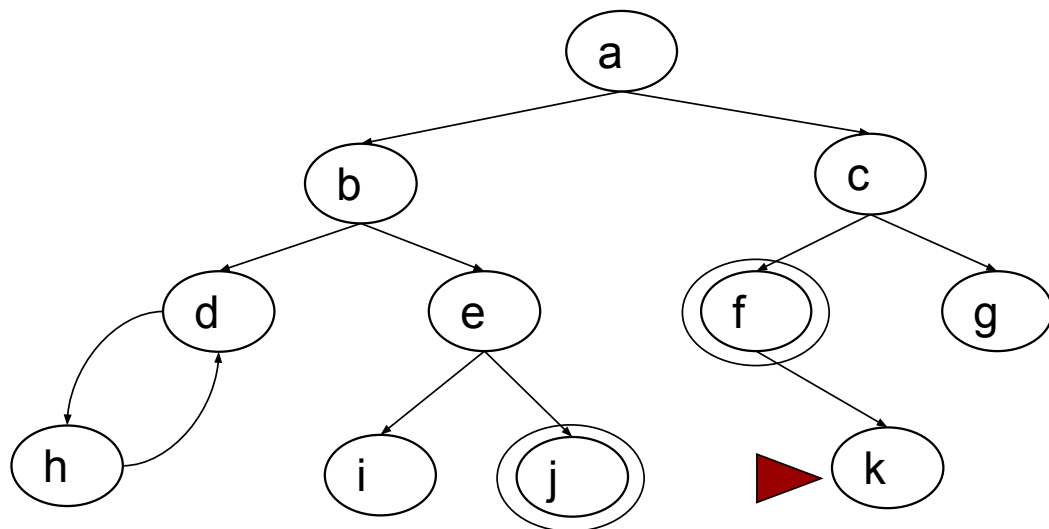
Busca em Largura



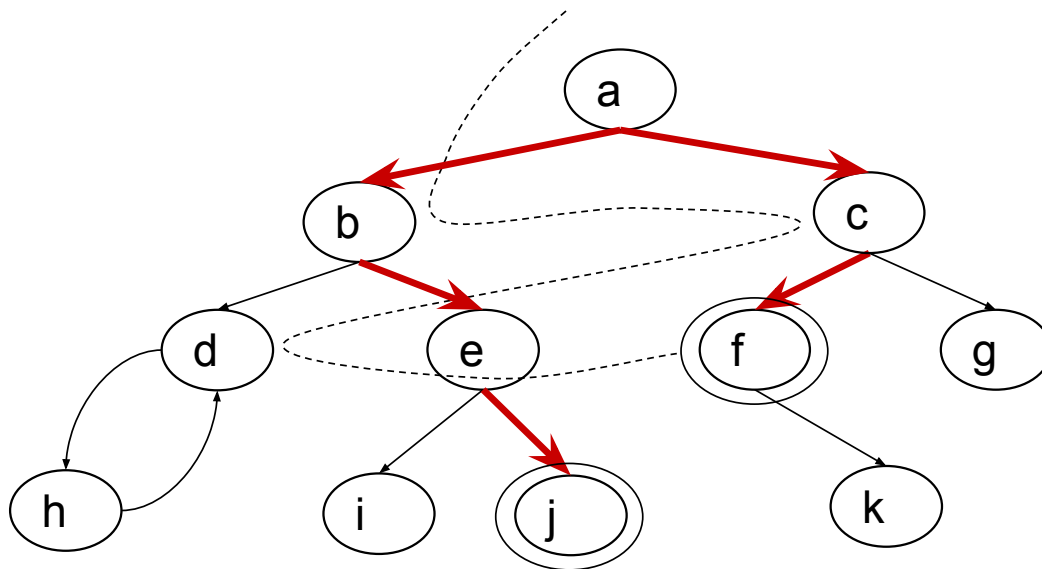
Busca em Largura



Busca em Largura



Busca em Largura



63

Busca em Largura

- (Breadth-First Search - BFS)
 - Nó objetivo é encontrado é sempre o mais raso, mas não necessariamente o ótimo.
 - Exceto se o custo for igual para todas as ações.
 - Ideal para quando o custo do caminho for uma função crescente da profundidade do nó

64



Busca em Largura

- Custo de busca
 - Supondo que o fator de ramificação da árvore seja b , temos para cada nível:
■ b + b^2 + b^3 + ... + b^d = $O(b^d)$
- Complexidade de espaço
 - Cada nó gerado permanece em memória
■ $O(b^d)$



Busca em Largura

- Custo de busca
 - Supondo que o fator de ramificação da árvore seja b , temos para cada nível:
■ b + b^2 + b^3 + ... + b^d = $O(b^d)$
- Complexidade de espaço
 - Cada nó gerado permanece em memória
■ $O(b^d)$





Busca em Largura

- Fator de ramificação: $b = 10$

Profundidade	Nós	Tempo	Memória
2	110	0,11 milissegundo	107 KB
4	11.110	11 milissegundos	10,6 MB
6	10^6	1,1 segundo	1GB
16	10^{16}	350 anos	10 EB

Busca em Profundidade

- (Depth-First Search - DFS)
- Implementação usando LIFO
- Expansão do nó mais profundo na borda atual
- Se um nó for expandido e a solução não estiver no seus descendentes, então ele é removido da memória.
- Se um nó possui infinitos descendentes, sendo que nenhum possui a solução, então a execução é infinita.

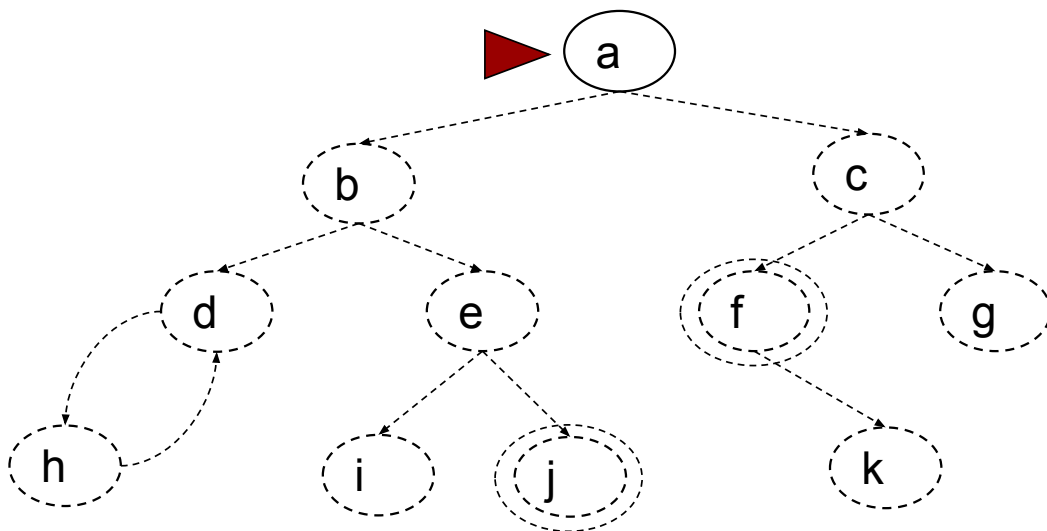




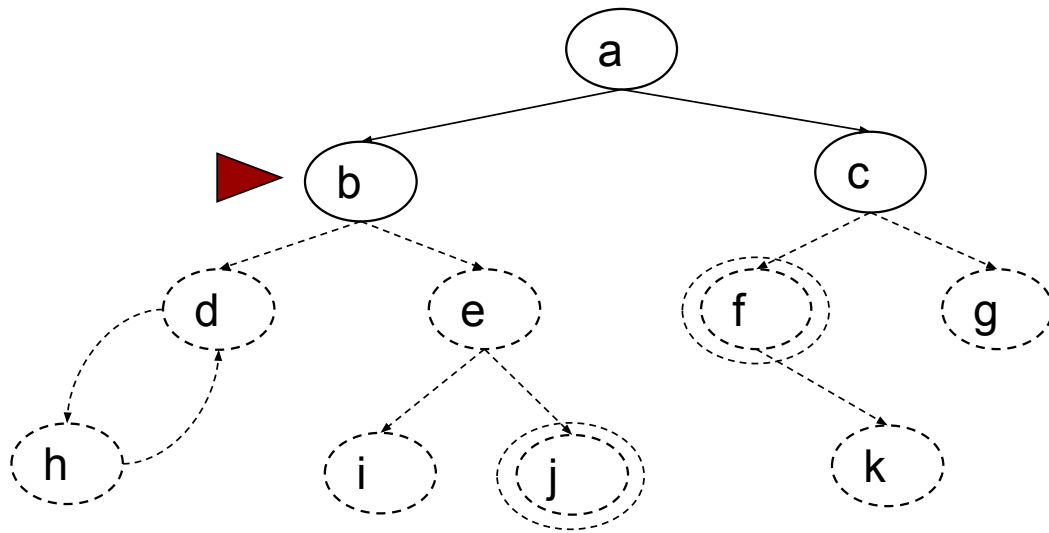
Busca em Profundidade

- Complexidade temporal
 - $O(b^m)$
 - m pode ser maior que d
- Complexidade de espaço
 - $O(bm)$
- Variação busca com retrocesso
 - Apenas um sucessor é gerado por vez
 - $O(m)$

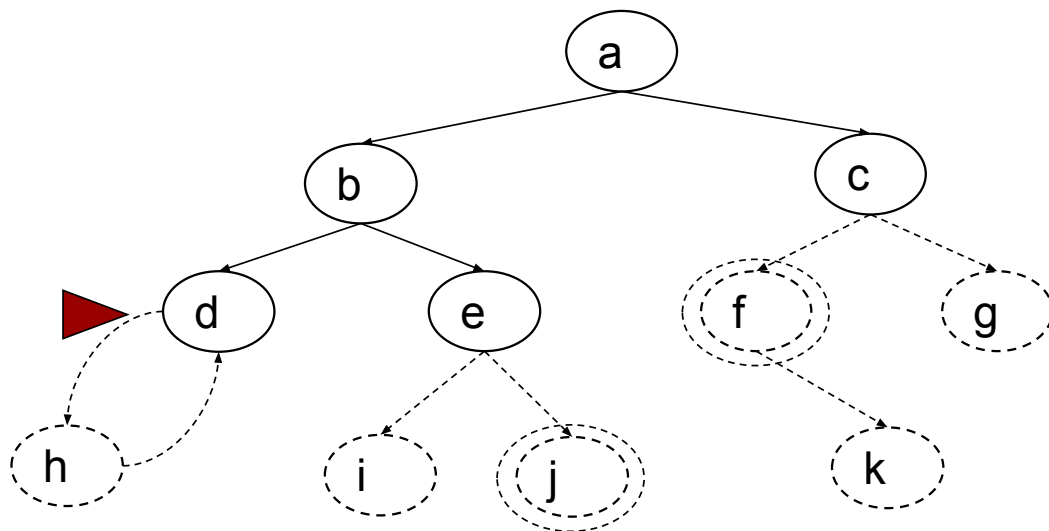
Busca em Profundidade



Busca em Profundidade



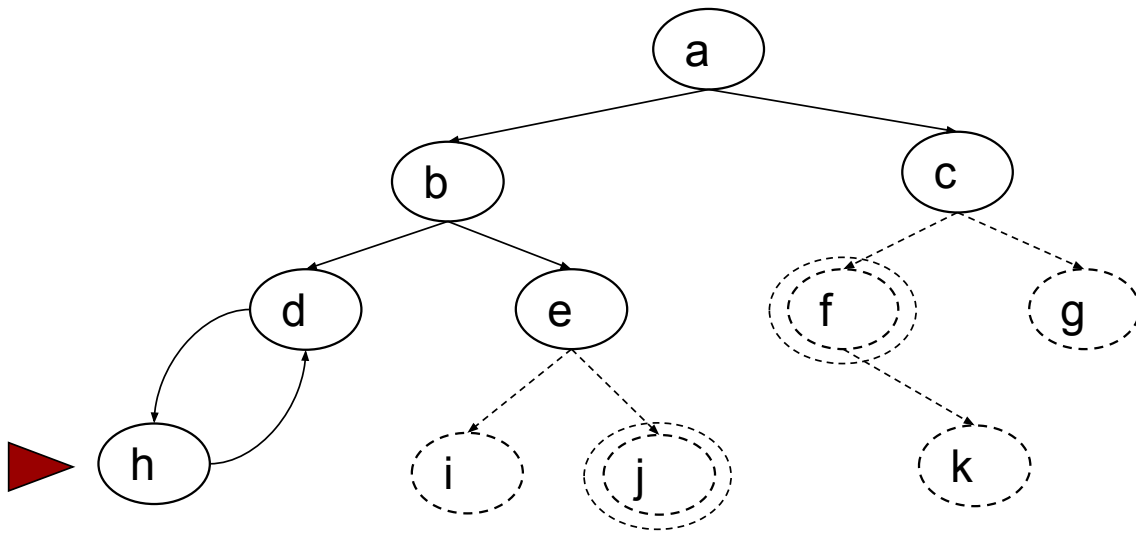
Busca em Profundidade



Busca em Profundidade



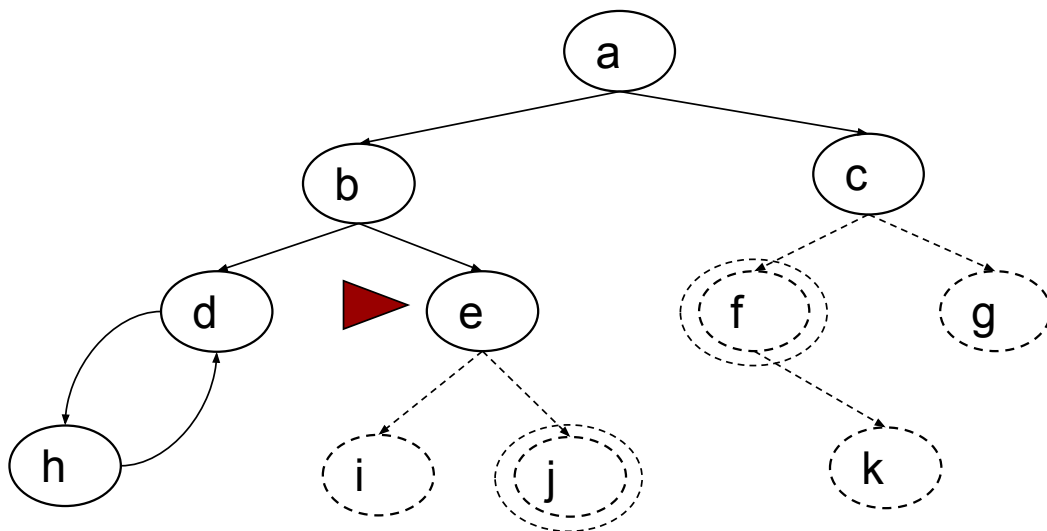
73



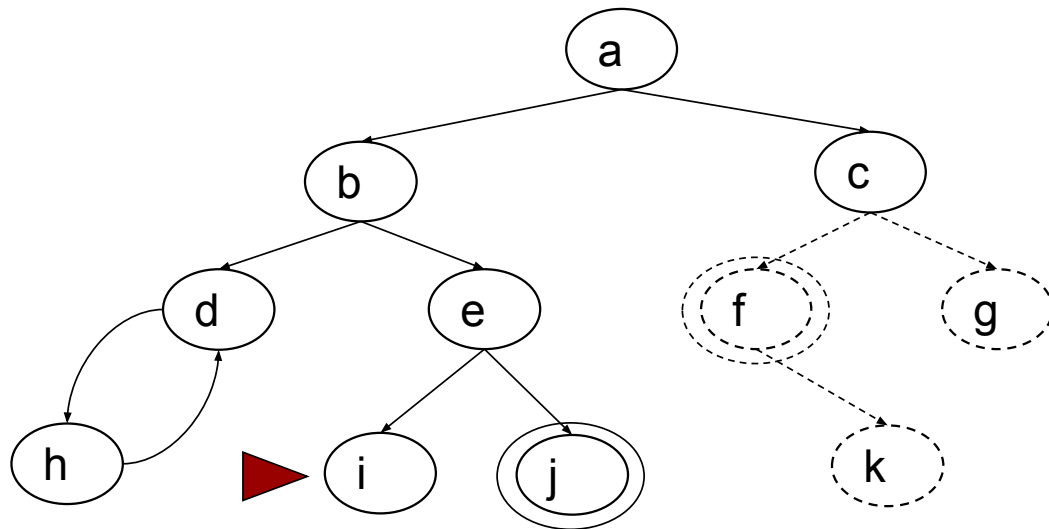
Busca em Profundidade



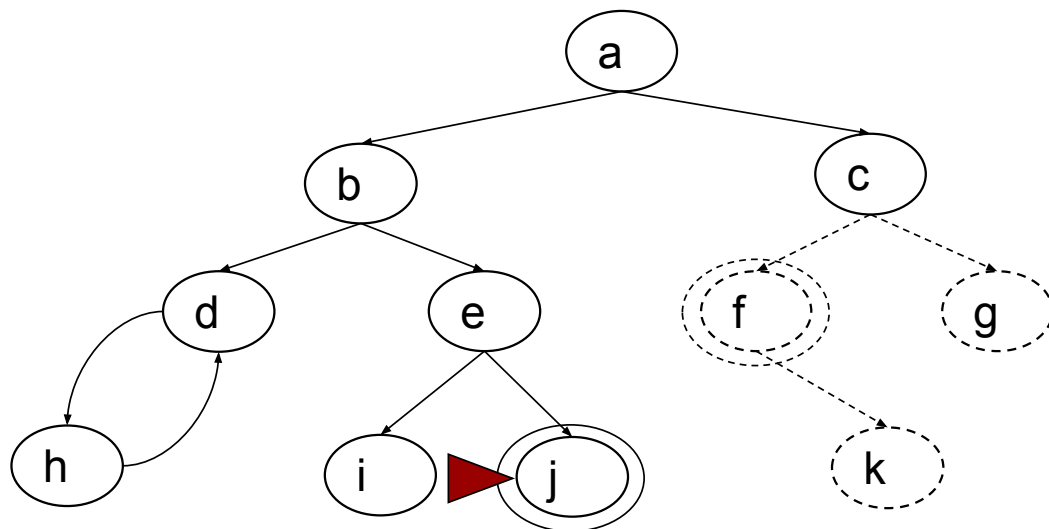
74



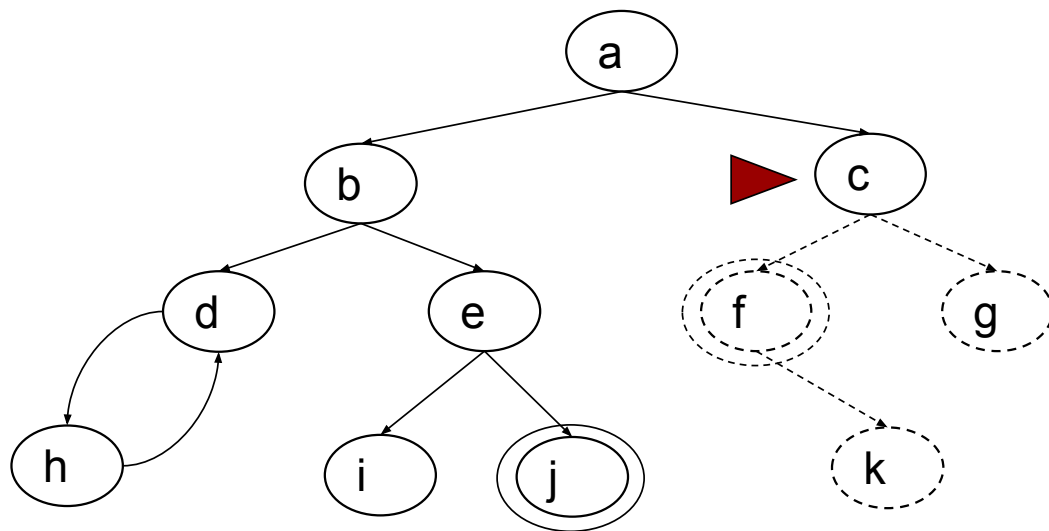
Busca em Profundidade



Busca em Profundidade

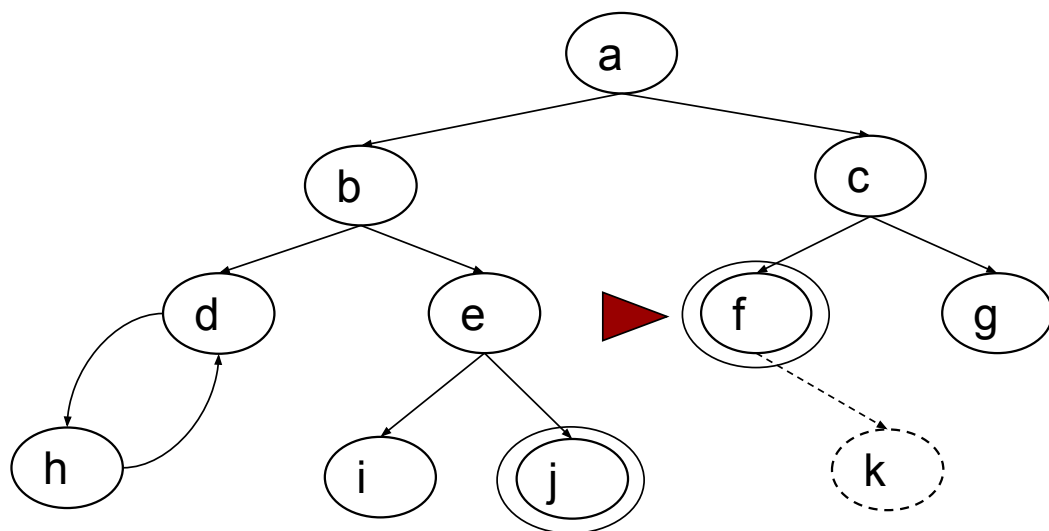


Busca em Profundidade



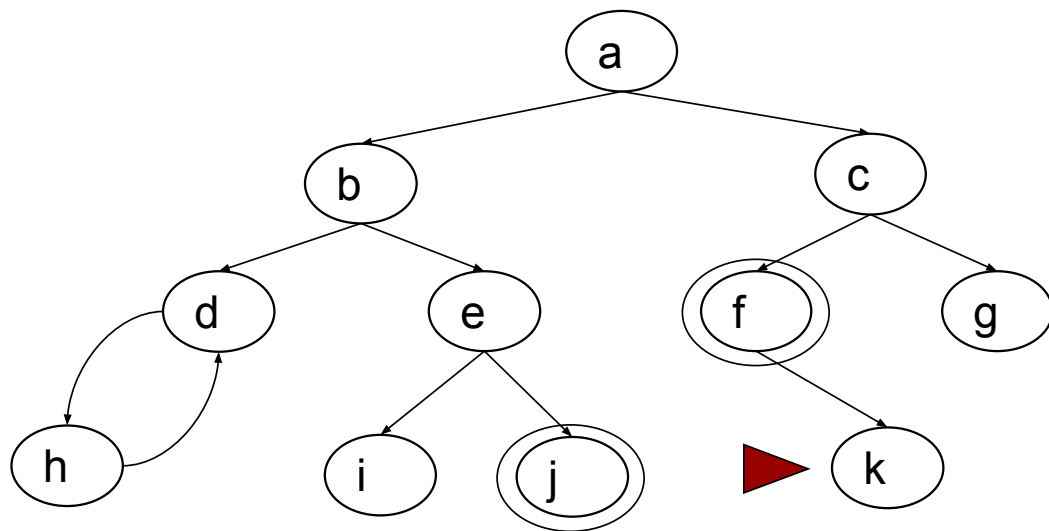
77

Busca em Profundidade

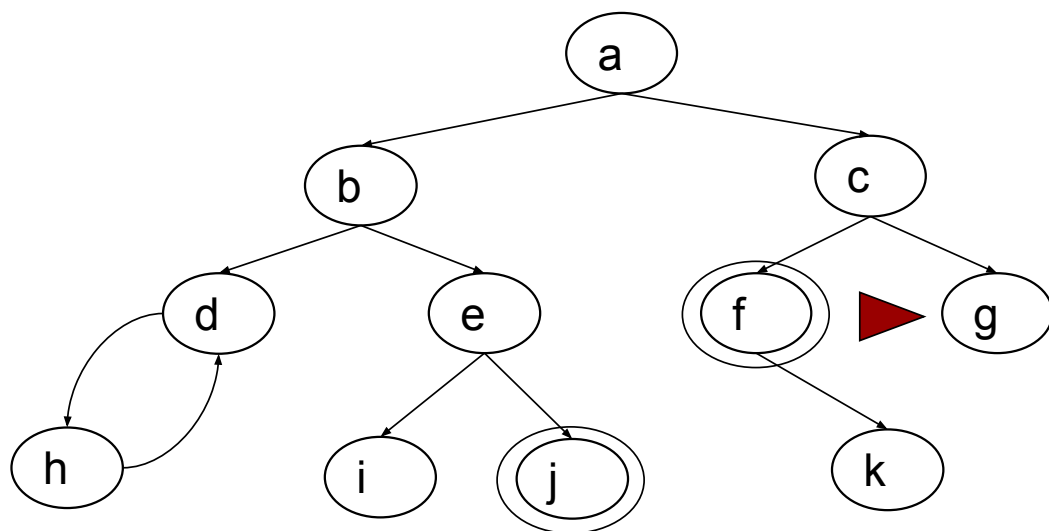


78

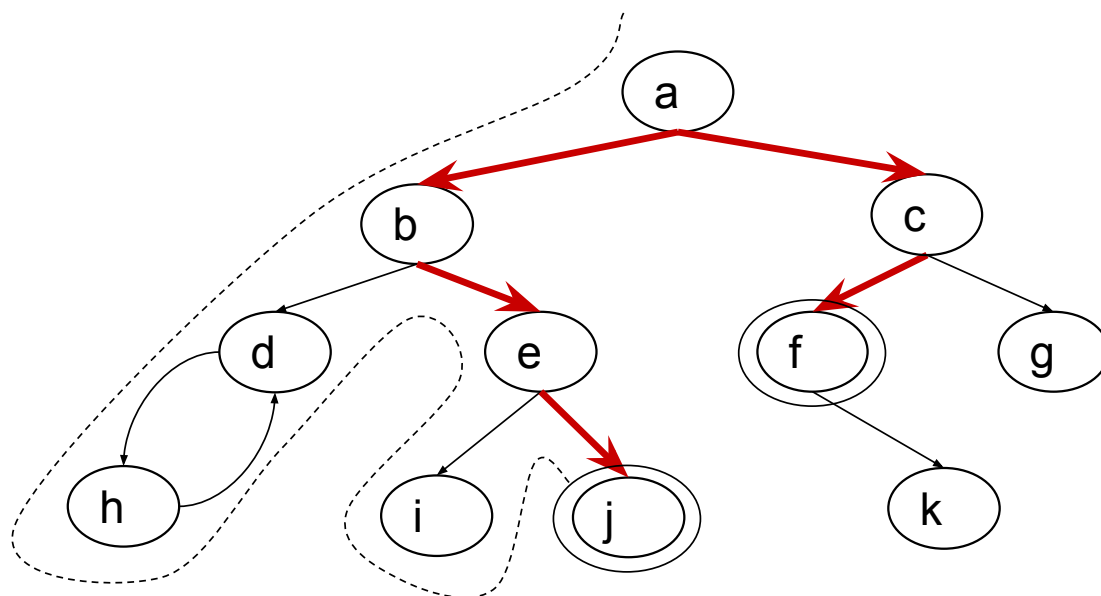
Busca em Profundidade



Busca em Profundidade

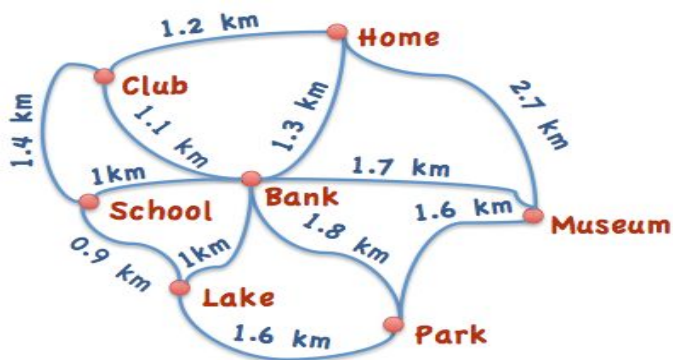


Busca em Profundidade



Exercício

- Considere “School” como o estado inicial e “Museum” como o estado final buscado. Monte a árvore de busca gerada pelo algoritmo de busca em profundidade.





Exercício

- Existem dois jarros inicialmente vazios. Um possui capacidade igual a 3 litros e o outro igual a 4 litros. Ambos podem ser enchidos completamente utilizando uma torneira e podem também ser esvaziados, despejando a água em um ralo. Além disso, a água presente em um jarro pode ser passada para o outro. Os jarros não possuem marcações e não é permitido o uso de qualquer instrumento de medida. Deseja-se colocar exatamente dois litros de água no jarro maior. Pode-se representar as quantidades de água presentes nos dois jarros pelo par ordenado (x,y) , em que x é a quantidade de água no jarro menor e y a quantidade no jarro maior. O estado inicial (ambos vazios) é o par $(0,0)$ e o objetivo é encontrar um par do tipo $(x,2)$, isto é, dois litros no jarro maior e qualquer quantidade no jarro menor.

Exercício

- Solucione este problema utilizando os algoritmos de busca em profundidade e em largura.
- Você pode implementar sua solução ou usar aplicativos para facilitar o entendimento dos algoritmos de busca

<http://www.aispace.org/search/search.jnlp>

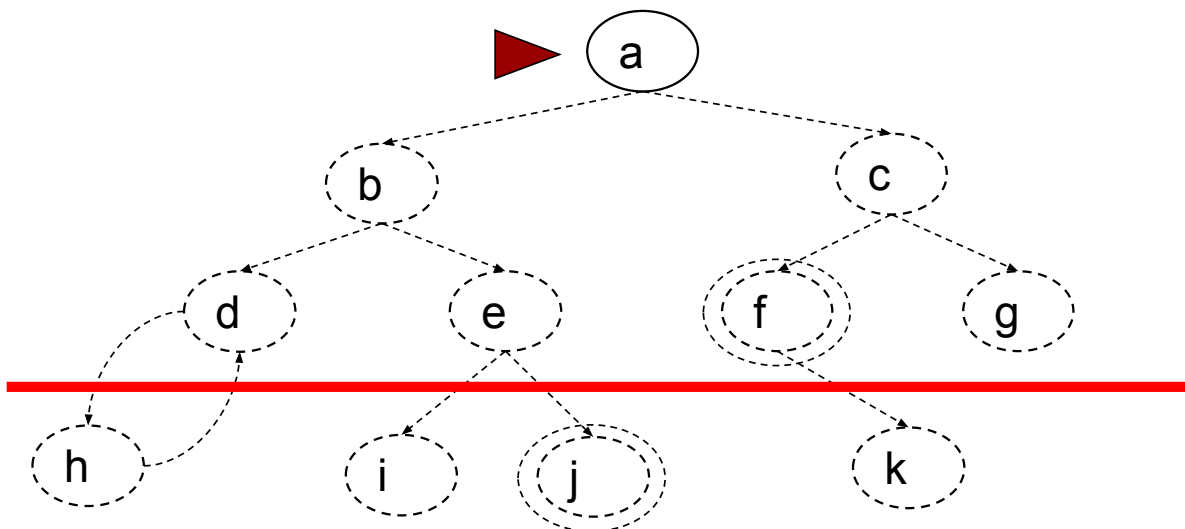




Busca em Profundidade Limitada

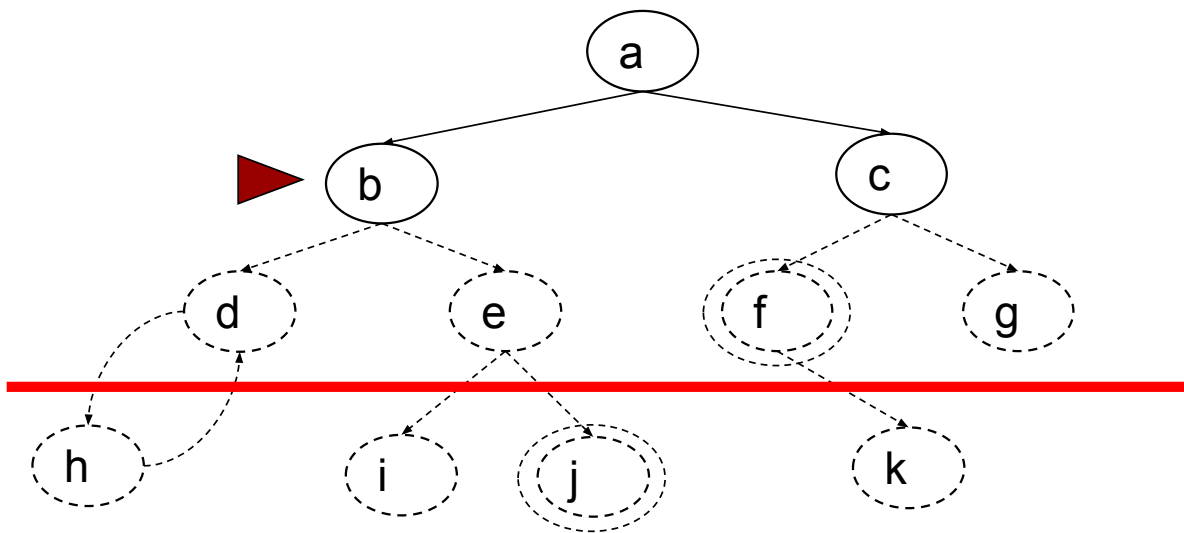
- Determina uma profundidade máxima (l) que o algoritmo pode ir.
- Complexidade de tempo
 - $O(b^l)$
- Complexidade de espaço
 - $O(bl)$

Busca em Profundidade Limitada ($L=2$)



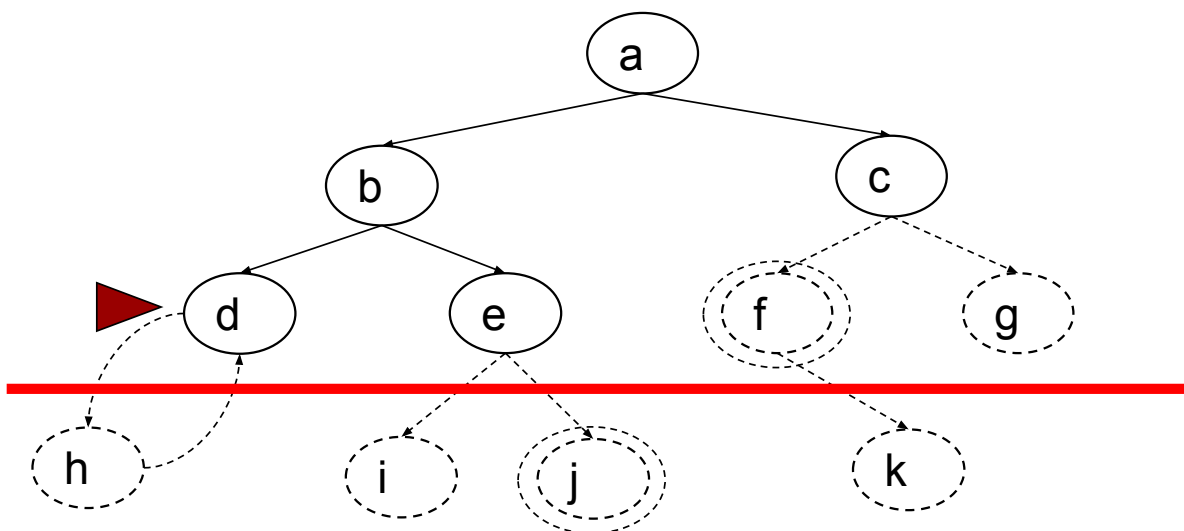


Busca em Profundidade Limitada (L=2)



87

Busca em Profundidade Limitada (L=2)



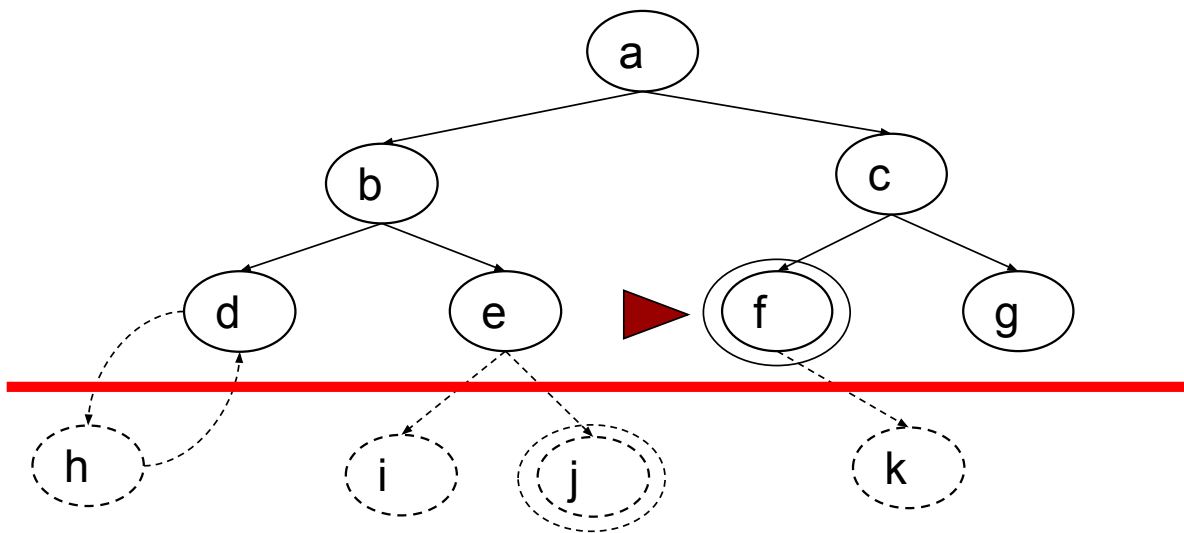
88





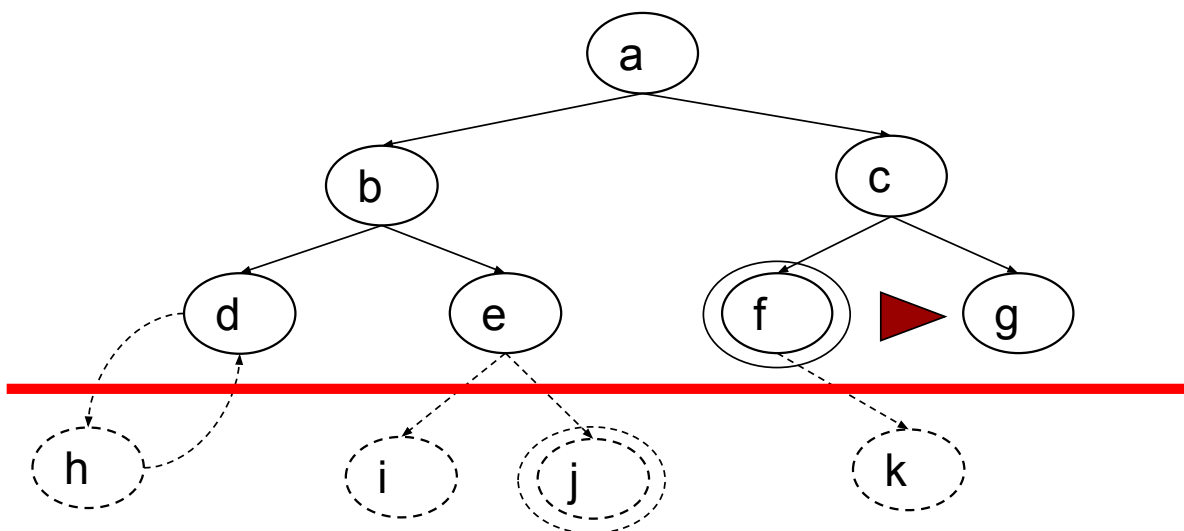


Busca em Profundidade Limitada (L=2)



91

Busca em Profundidade Limitada (L=2)



92



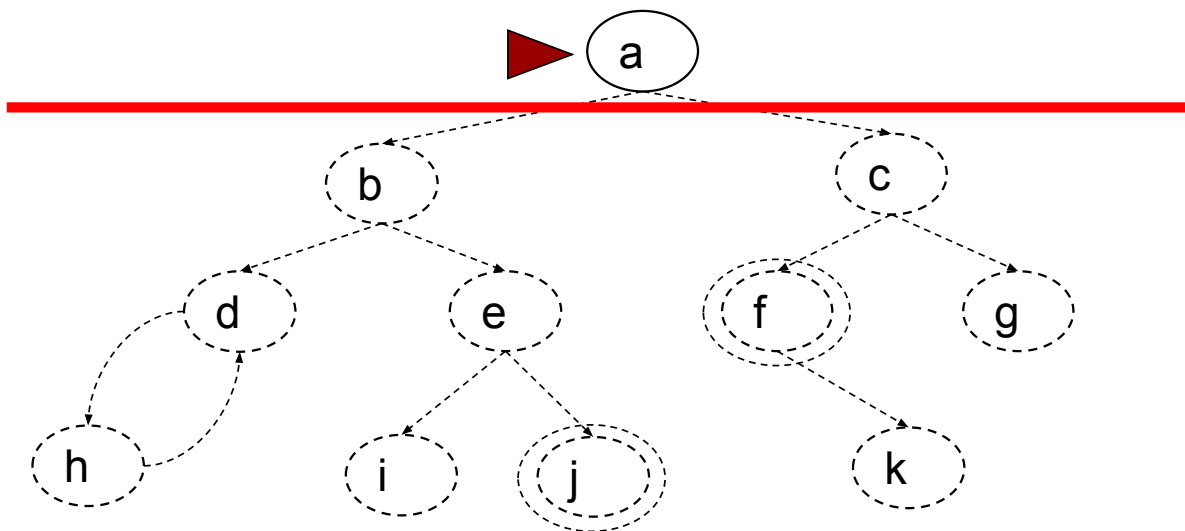


Busca de Aprofundamento Iterativo

- (Iterative Deepening Search - IDS)
- Permite encontrar o melhor limite de profundidade

93

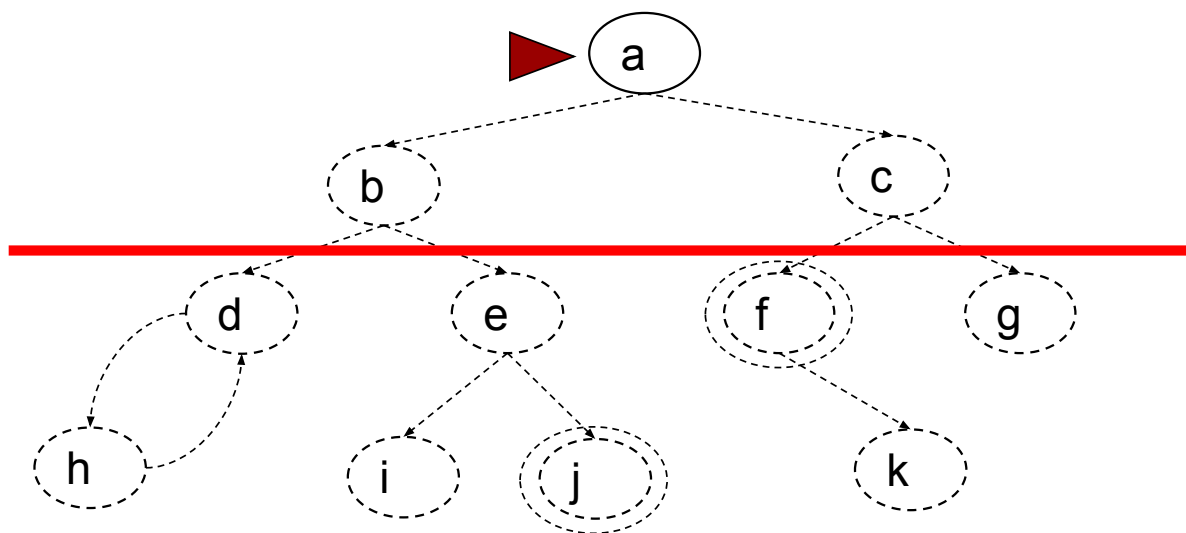
IDS (L=0)



94

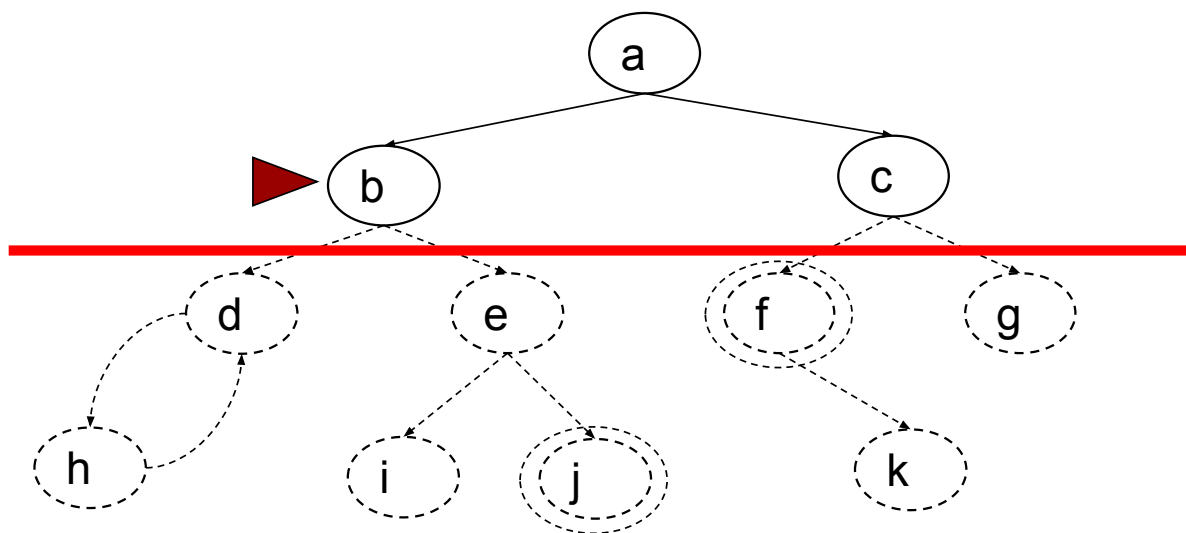


IDS (L=1)



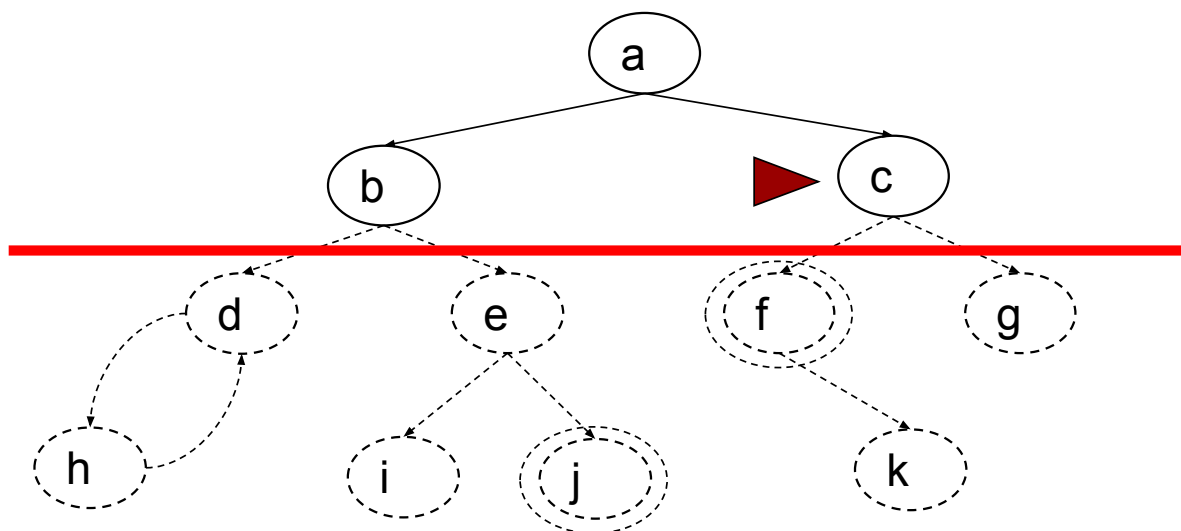
95

IDS (L=1)



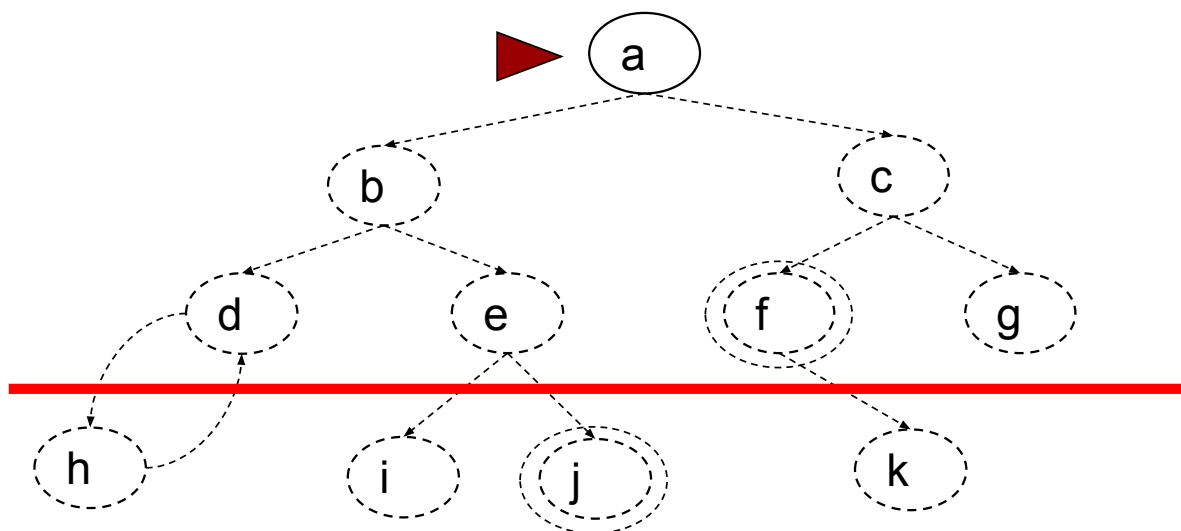
96

IDS (L=1)



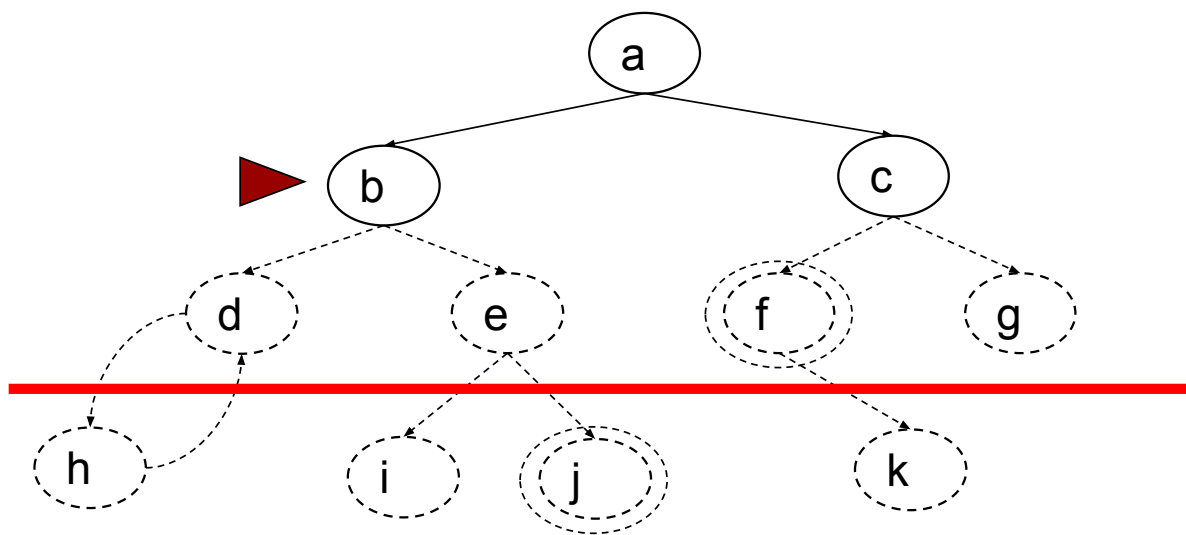
97

IDS (L=2)

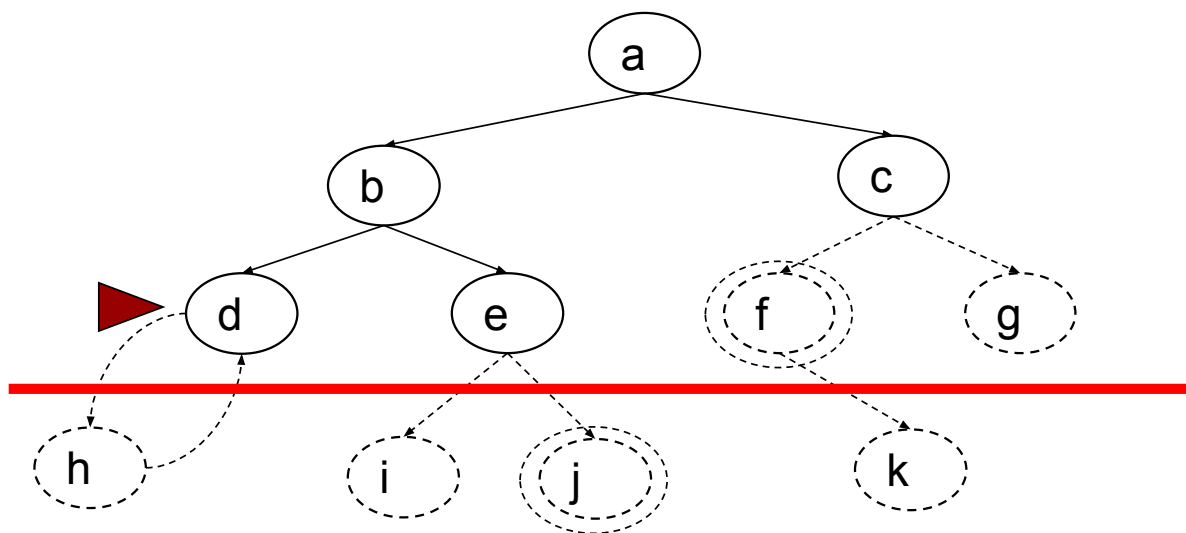


98

IDS (L=2)

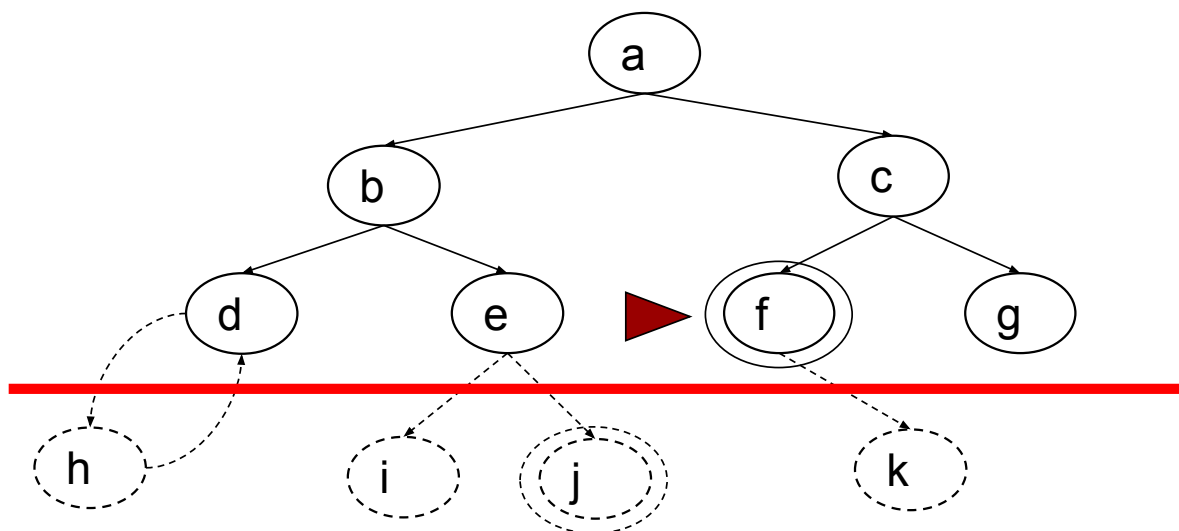


IDS (L=2)

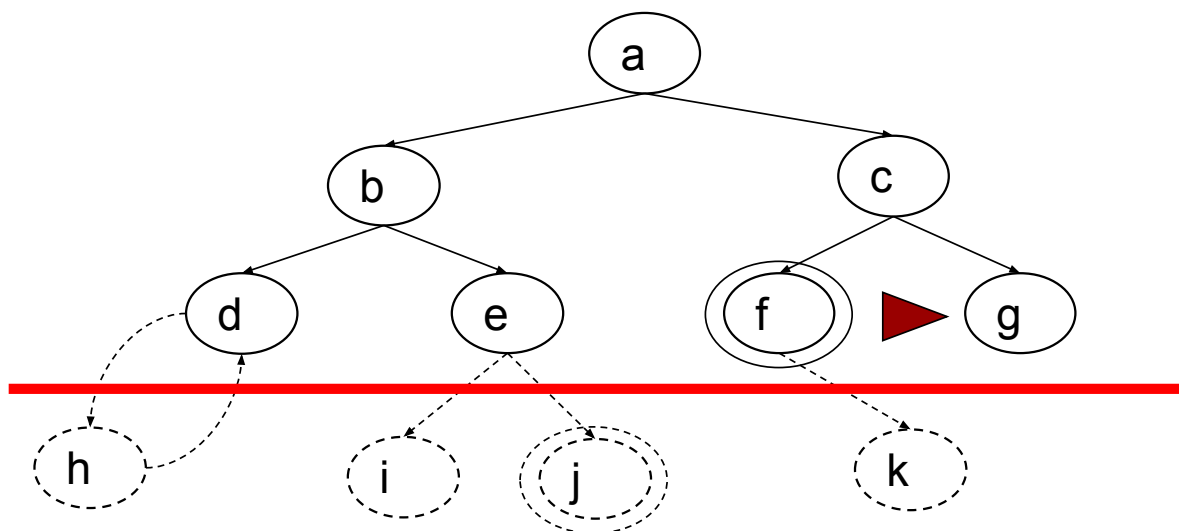




IDS (L=2)



IDS (L=2)





Busca bidirecional

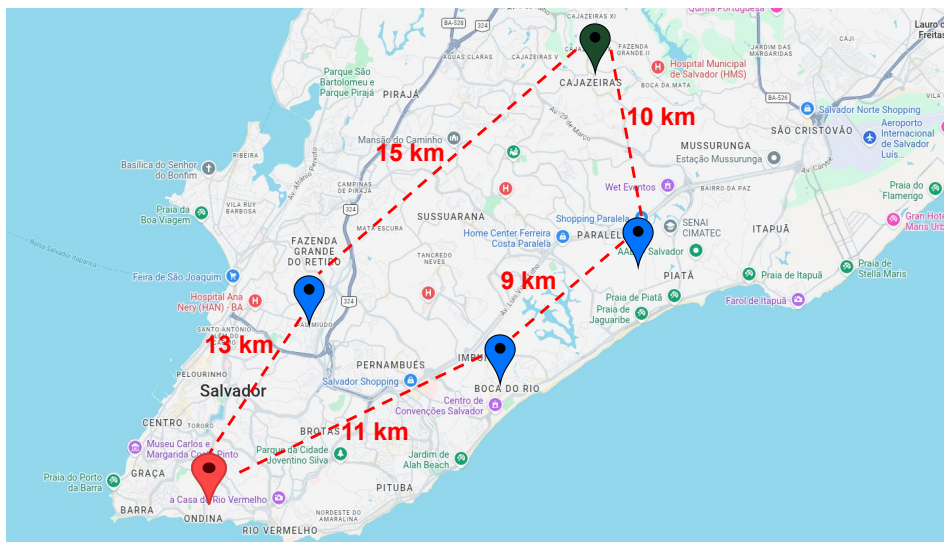
- Execução de duas buscas simultâneas
 - Direta: partindo do estado inicial
 - Inversa: partindo do objetivo
 - $O(b^{d/2} + b^{d/2}) \ll O(b^d)$
 - Objetivo: quando as bordas das duas buscas se cruzam

Busca de Custo Uniforme

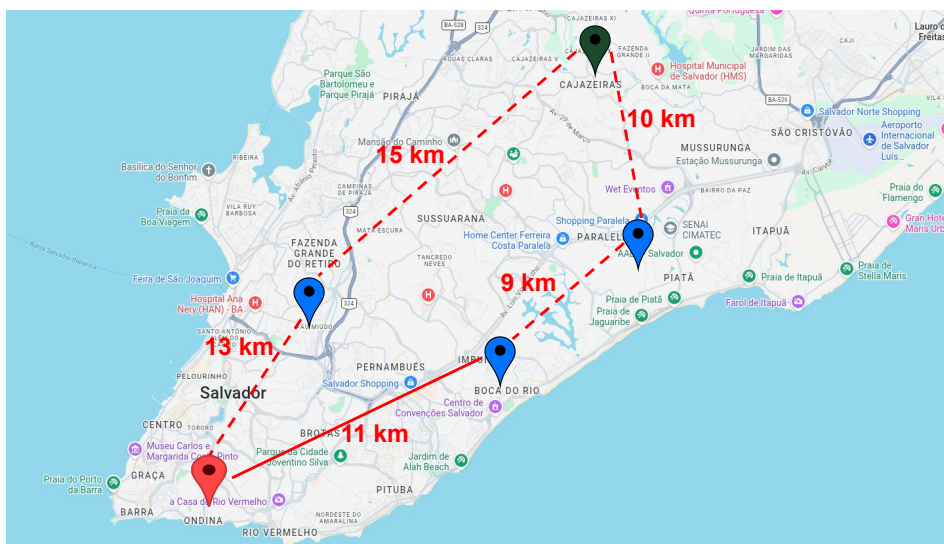
- Quando todos os custos forem iguais, BFS é ótima por expandir sempre o nó mais raso.
 - BRANCH-AND-BOUND
- Com qualquer valor de custo, Busca de Custo Uniforme:
 - Ao invés de expandir o nó mais raso, deve-se expandir o nó com a menor **função de custo: $g(n)$** .
- Armazenamento da borda como fila de prioridade.



Busca de Custo Uniforme



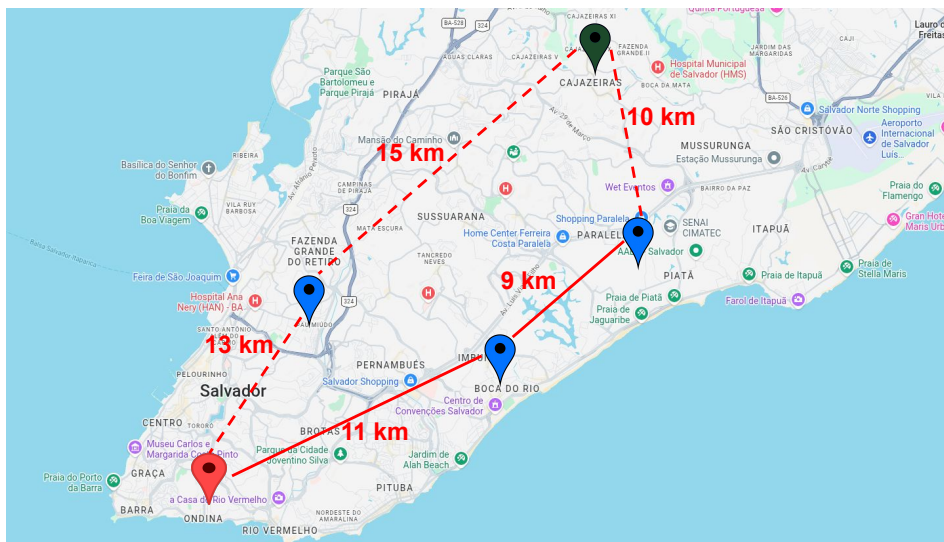
Busca de Custo Uniforme



(1) Ondina → Boca do Rio (11 km)

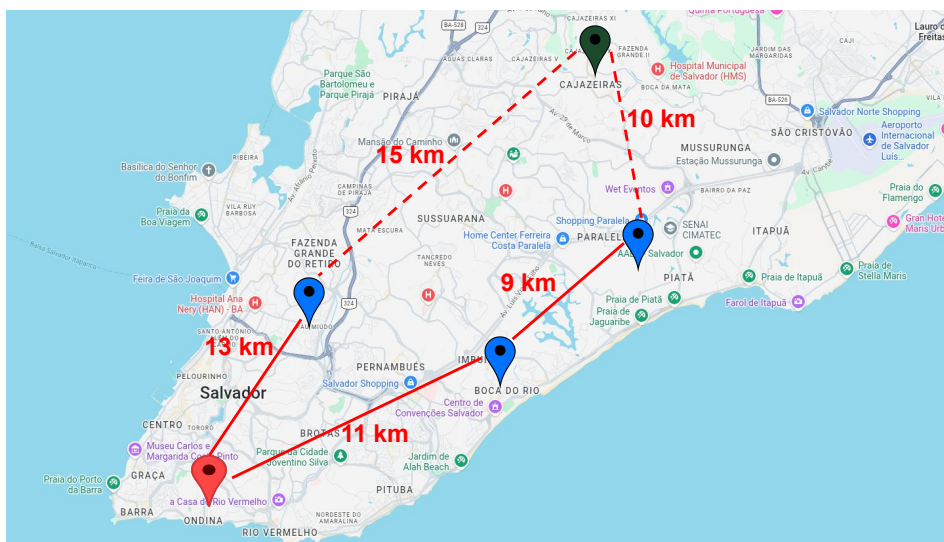


Busca de Custo Uniforme



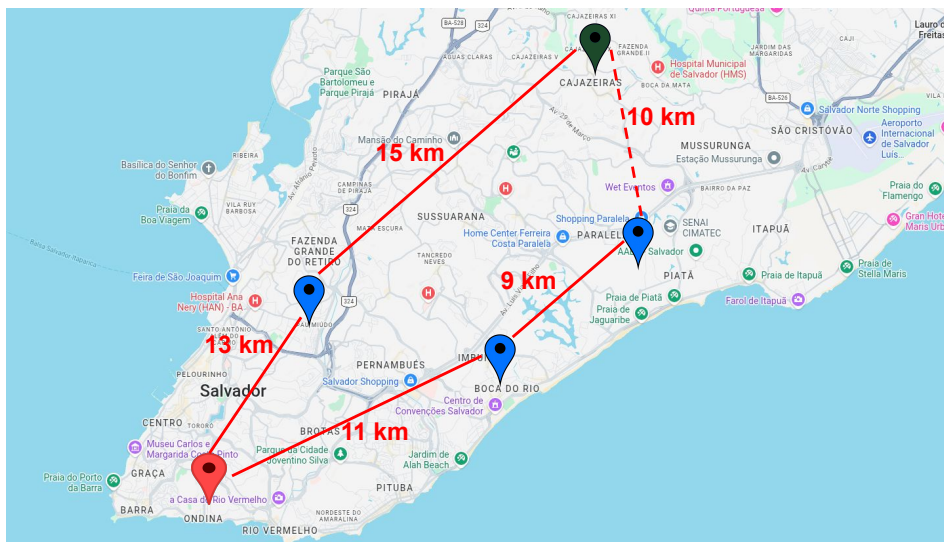
- (1) Ondina → Boca do Rio (11 km) → Piatã (20 km)

Busca de Custo Uniforme



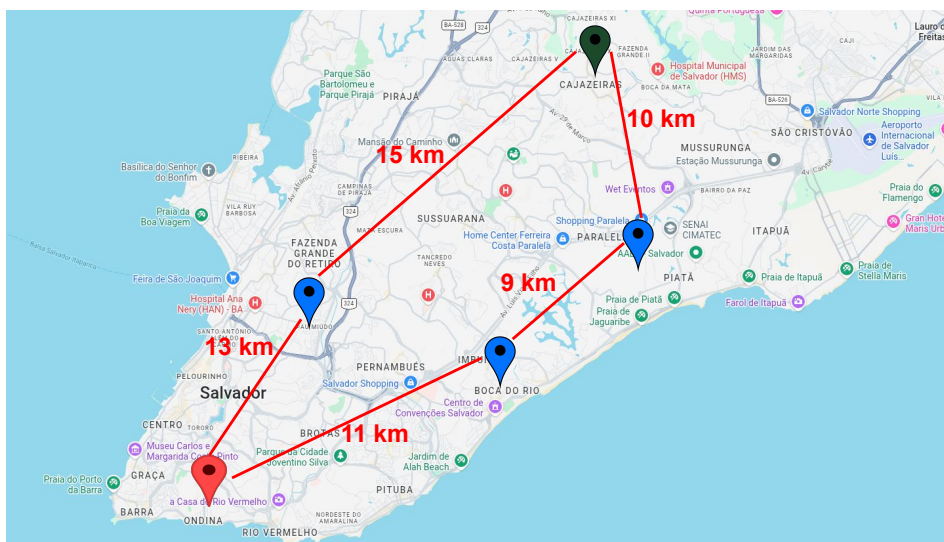
- (1) Ondina → Boca do Rio (11 km) → Piatã (20 km)
- (2) Ondina → Faz. Grande (13 km)

Busca de Custo Uniforme



- (1) Ondina → Boca do Rio (11 km) → Piatã (20 km)
- (2) Ondina → Faz. Grande (13 km) → Cajazeiras (28 km)

Busca de Custo Uniforme



- (1) Ondina → Boca do Rio (11 km) → Piatã (20 km) → Cajazeiras (30 km)
- (2) Ondina → Faz. Grande (13 km) → Cajazeiras (28 km)



Busca de Custo Uniforme

- Expande os nós na ordem do custo de caminho ótimo
- Adaptação usando uma fila ordenada pelo custo
- O primeiro nó objetivo selecionado será a solução ótima
- Não se preocupa com a quantidade de passos, mas com o custo total
- Se o custo de todos os passos são iguais, então ela é igual a BFS
- Seja C^* o custo da solução ótima e cada ação tenha custo ϵ
 - $O(b^{1+\lceil C^*/\epsilon \rceil})$
 - Pode ser maior que $O(b^d)$

113



Busca Heurística (informada)

114



Busca Heurística

- Digamos que você está numa Cidade, e quer pegar um trem para casa, mas não sabe qual deve pegar.
 - Se você morasse na zona Norte, naturalmente ignoraria todos os trens que fossem para o sul.
 - Se você morasse na zona Sul, naturalmente ignoraria todos os trens que fossem para o Norte.

Busca Heurística

- A busca em grafos pode atingir uma complexidade elevada devido ao número de alternativas
- Estratégias de busca informada utilizam informação heurística sobre o problema para encurtar a busca no espaço de estados
- Essa estimativa indica o quanto o nó é promissor com relação a atingir a meta estabelecida
- Utiliza informações específicas do domínio para ajudar na decisão





Busca Heurística

- O principal papel da heurística é eliminar, ou podar, ramos da busca.
- A tradução de uma heurística para um valor numérico é realizado pela **função de avaliação: $h(n)$** .
- $g(n) \rightarrow$ custo de um caminho que sai de um nó i até o nó n .
- $h(n) \rightarrow$ custo estimado do caminho de menor custo do nó n até o nó objetivo.



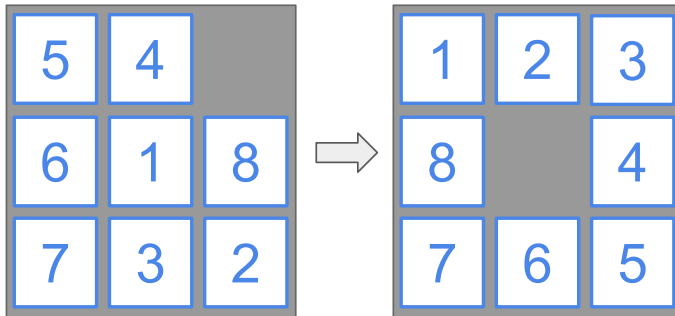
Busca Heurística

- O principal papel da heurística é eliminar, ou podar, ramos da busca.
- A tradução de uma heurística para um valor numérico é realizado pela **função de avaliação: $h(n)$** .
- Exemplo:
 - Como determinar uma função de avaliação para o problema 8-puzzle?



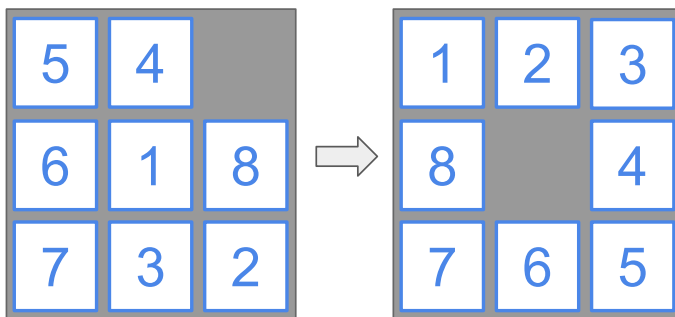
Busca Heurística

- Número de peças fora do lugar, ignorando a posição vazia



Busca Heurística

- Número de peças fora do lugar, ignorando a posição vazia



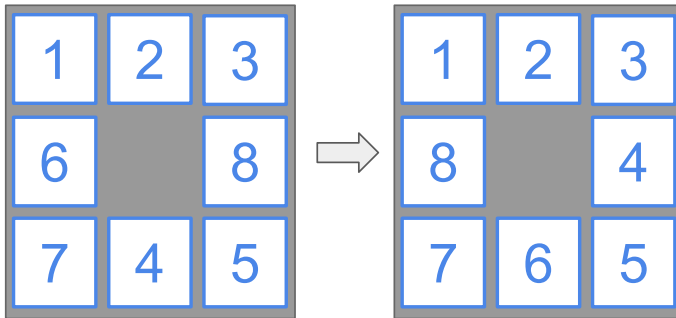
- Existem 8 peças fora do lugar, logo: $h(n) = 8$





Busca Heurística

- Distância de Manhattan (sem considerar espaços vazios)



- $h(n) = 6$

Busca Gulosa

- Busca gulosa de melhor escolha
 - Expande o nó que está mais próximo do objetivo.
 - Objetivo: encontrar a solução mais rapidamente.
 - Encontra uma solução sem nunca expandir um nó que não estiver no caminho
 - Exemplo de $h(\cdot)$:
 - $h_{DLR} \rightarrow$ Distância em Linha Reta

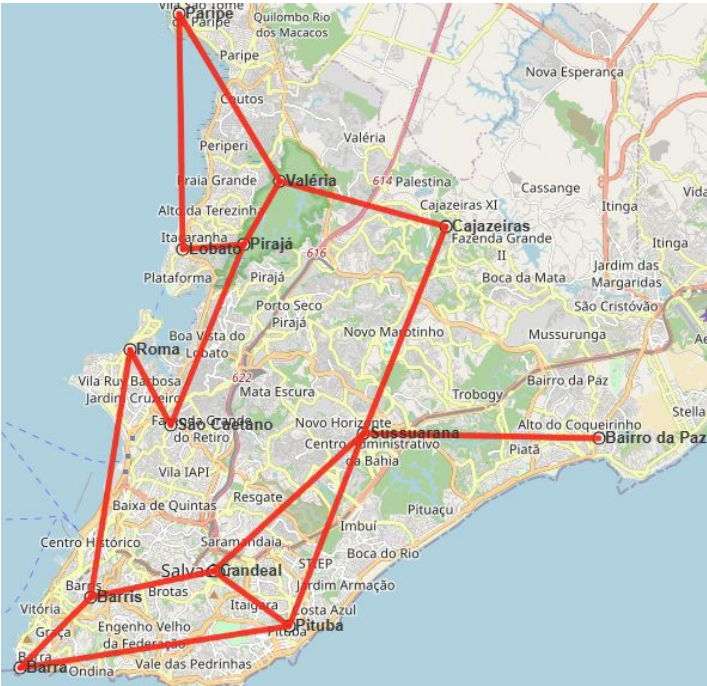


Exercício

- Uma empresa de entregas com drones autônomos precisa seguir rotas aéreas previamente definidas. Por questões de segurança, voos não podem conectar diretamente os pontos de origem e de destino quando a distância for muito grande. Essa empresa precisa pegar um produto na Barra e entregar em Valéria. Seguindo o mapa de vôo e as heurísticas apresentadas, qual seria a rota de entrega usando o algoritmo de Busca Gulosa?



Exercício



Bairro	$h(n)$
Barra	17
Pituba	13
Barris	14
Candeal	12
Roma	7
São Caetano	8
Sussuarana	8
Cajazeiras	5
Bairro da Paz	13
Pirajá	2
Lobato	4
Valéria	0
Paripe	6



Exercício

$h = 17$

Barra



Exercício

$h = 17$

Barra

$h = 14$

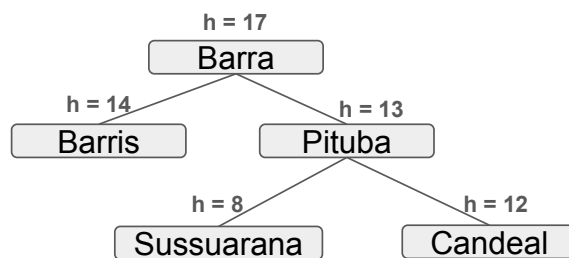
Barris

$h = 13$

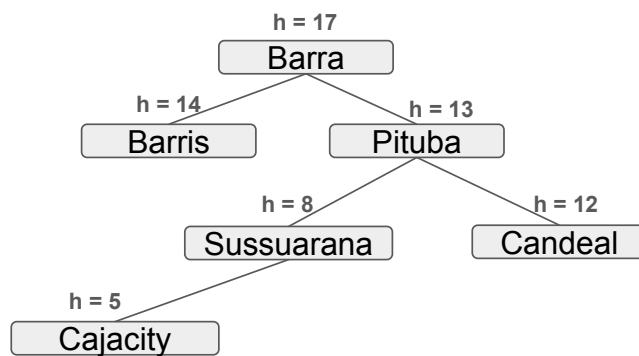
Pituba



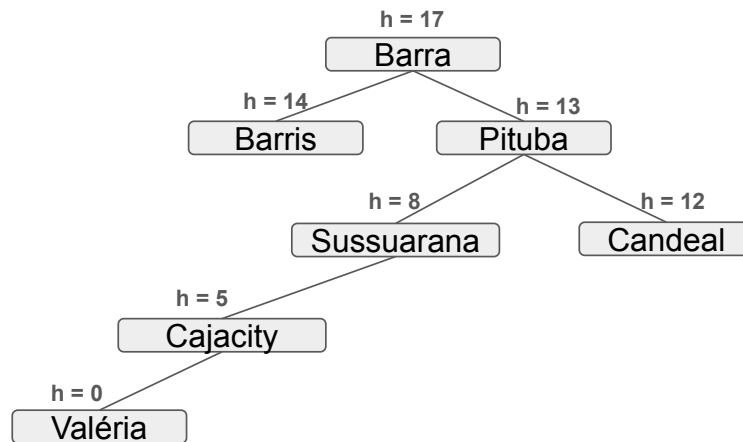
Exercício



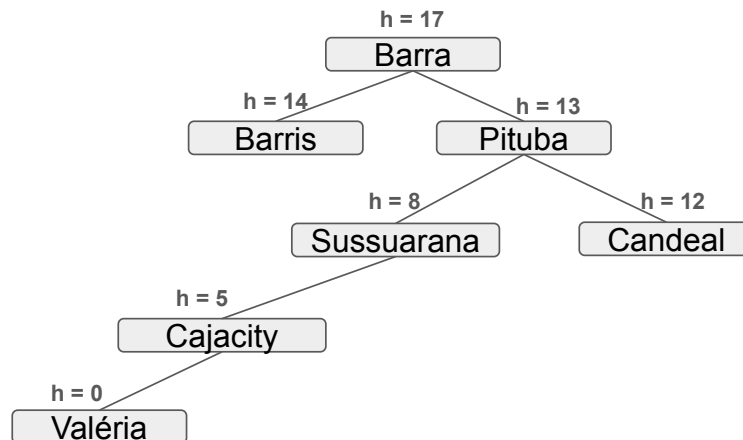
Exercício



Exercício

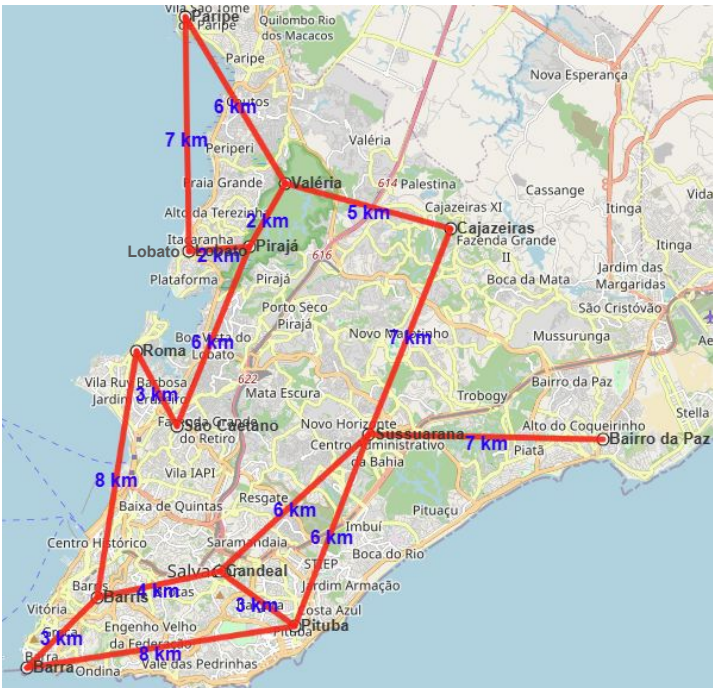


Exercício



Caminho: Barra → Pituba → Sussuarana → Cajazeiras → Valéria

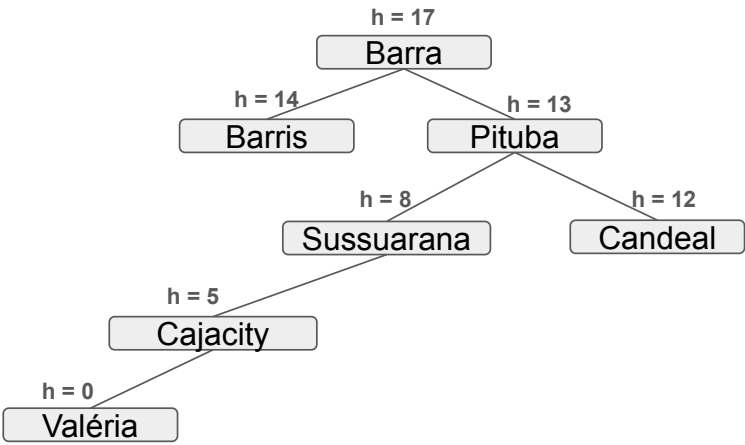
Exercício



Bairro	h(n)
Barra	17
Pituba	13
Barris	14
Candeal	12
Roma	7
São Caetano	8
Sussuarana	8
Cajazeiras	5
Bairro da Paz	13
Pirajá	2
Lobato	4
Valéria	0
Paripe	6



Exercício



Caminho: Barra → Pituba → Sussuarana → Cajazeiras → Valéria
8km + 6km + 7km + 5km = 26km





Busca Gulosa

- O caminho é ótimo?
- Assim como a Busca em Profundidade, também é incompleto!
 - Nunca aceita uma solução com $h(n)$ maior que o atual.
- Complexidade: $O(b^m)$
 - $m \rightarrow$ profundidade máxima no espaço de busca.

Algoritmo A*

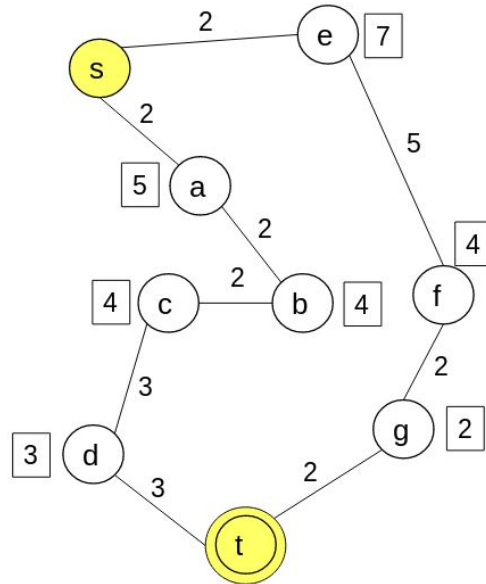
- Avalia um nó usando a seguinte função:
 - $f(n) = g(n) + h(n)$
 - $f(n)$ = custo estimado da solução de menor custo através de n .
- A* é completa e ótima.





Exercício

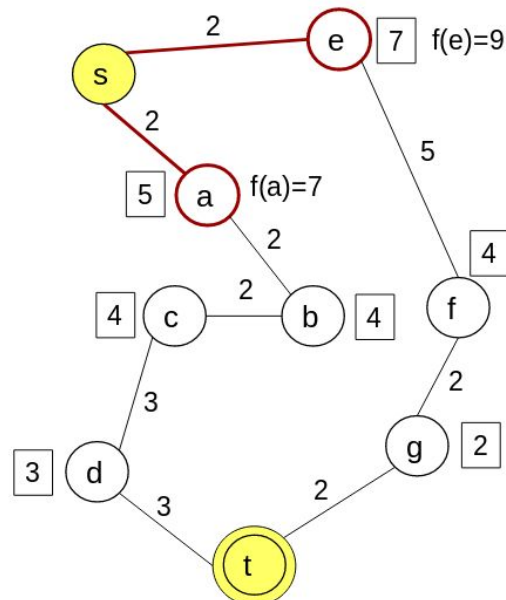
- Utilizando A^* encontre a solução saindo do estado S até o estado T



135

Exercício

- Utilizando A^* encontre a solução saindo do estado S até o estado T



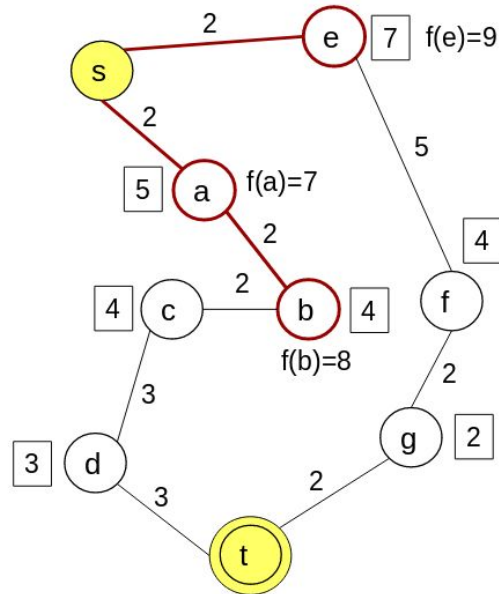
136





Exercício

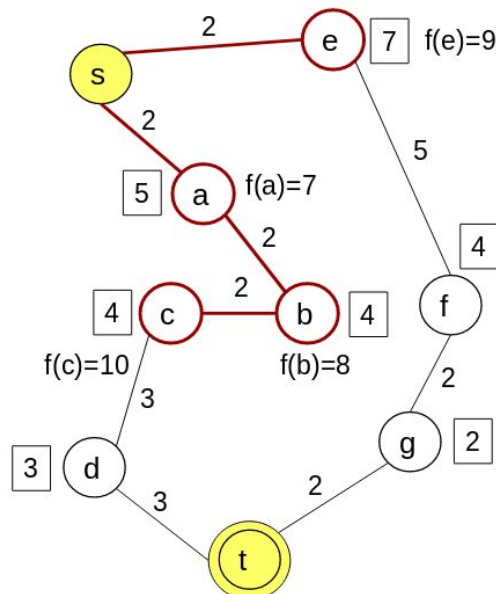
- Utilizando A^* encontre a solução saindo do estado S até o estado T



137

Exercício

- Utilizando A^* encontre a solução saindo do estado S até o estado T



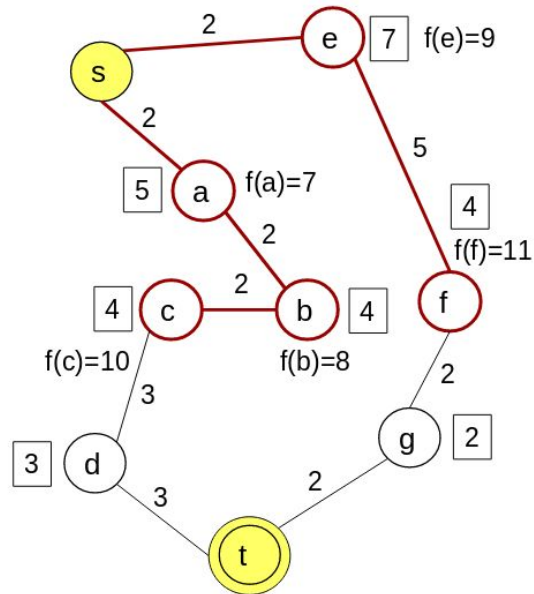
138





Exercício

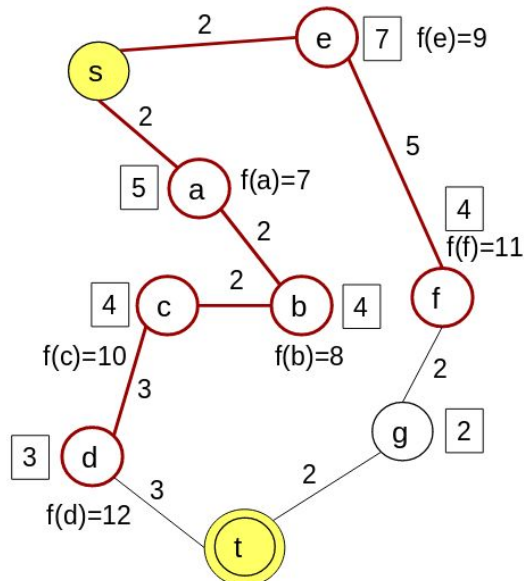
- Utilizando A^* encontre a solução saindo do estado S até o estado T



139

Exercício

- Utilizando A^* encontre a solução saindo do estado S até o estado T



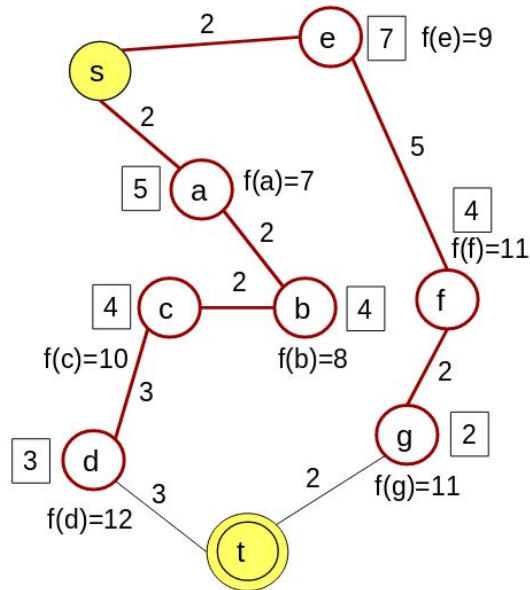
140





Exercício

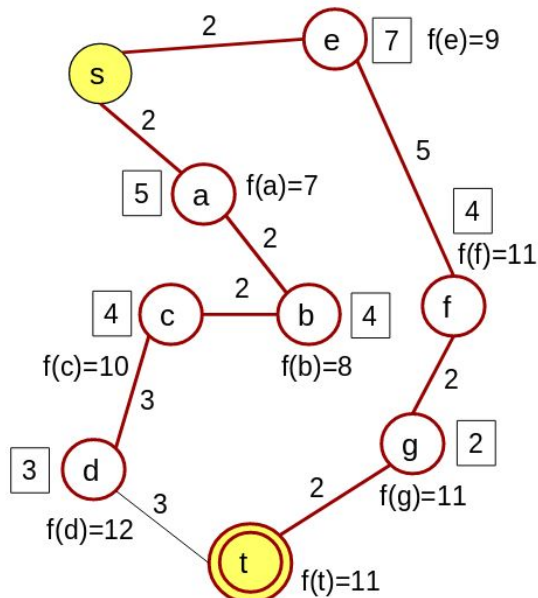
- Utilizando A^* encontre a solução saindo do estado S até o estado T



141

Exercício

- Utilizando A^* encontre a solução saindo do estado S até o estado T



142

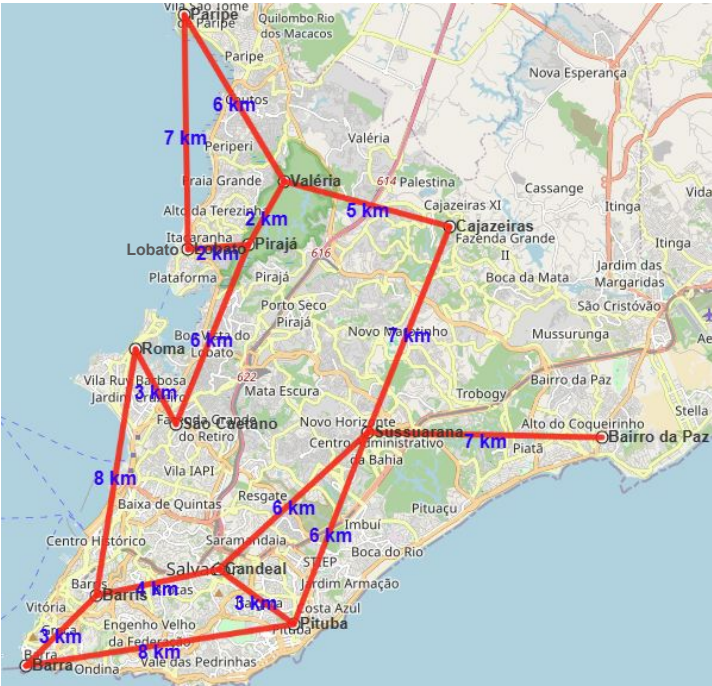


Exercício

- Refaça o exercício dos drones usando A*.



Exercício



Bairro	h(n)
Barra	17
Pituba	13
Barris	14
Candeal	12
Roma	7
São Caetano	8
Sussuarana	8
Cajazeiras	5
Bairro da Paz	13
Pirajá	2
Lobato	4
Valéria	0
Paripe	6



Exercício

$$f = 0 + 17 = 17$$

Barra



Exercício

$$f = 0 + 17 = 17$$

Barra

$$f = 3 + 14 = 17$$

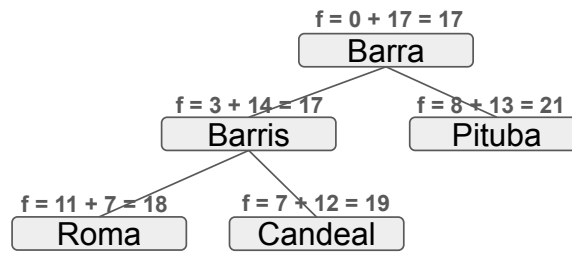
Barris

$$f = 8 + 13 = 21$$

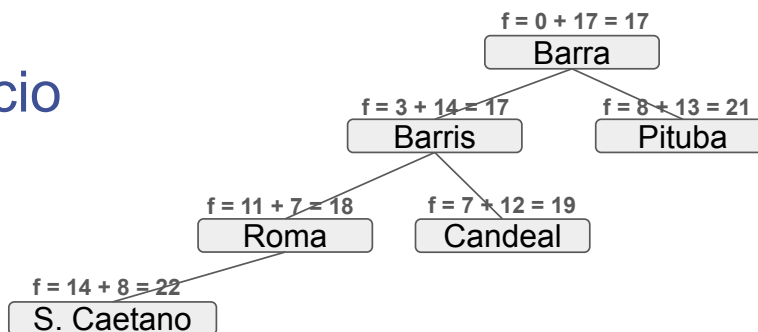
Pituba



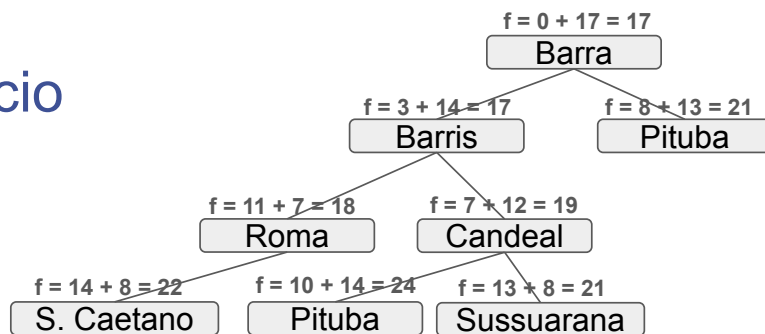
Exercício



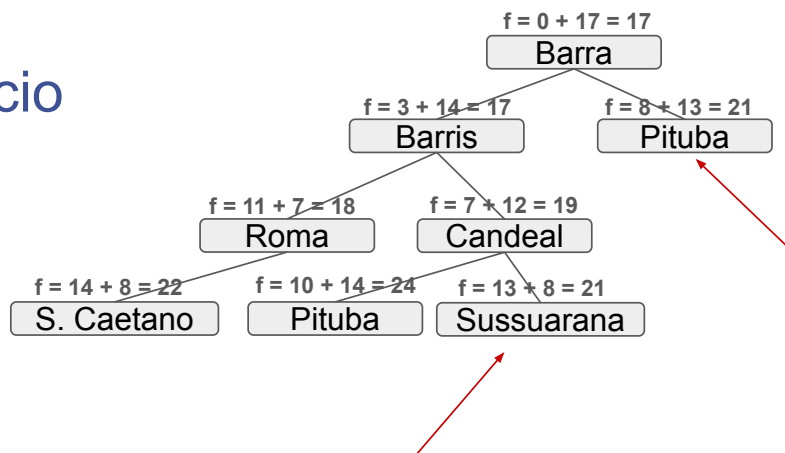
Exercício



Exercício



Exercício

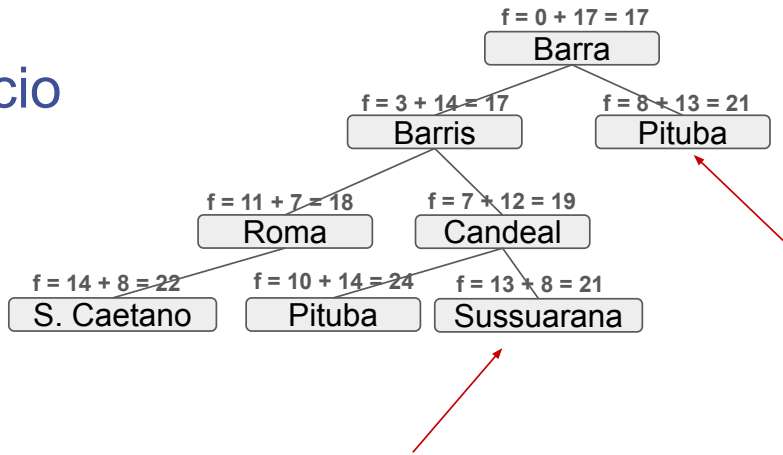


Empate! Como resolver?

- Ordem de chegada na Estrutura de dados (Pituba)
- Valor de $g(n)$
- Valor de $h(n)$
- Escolha aleatória



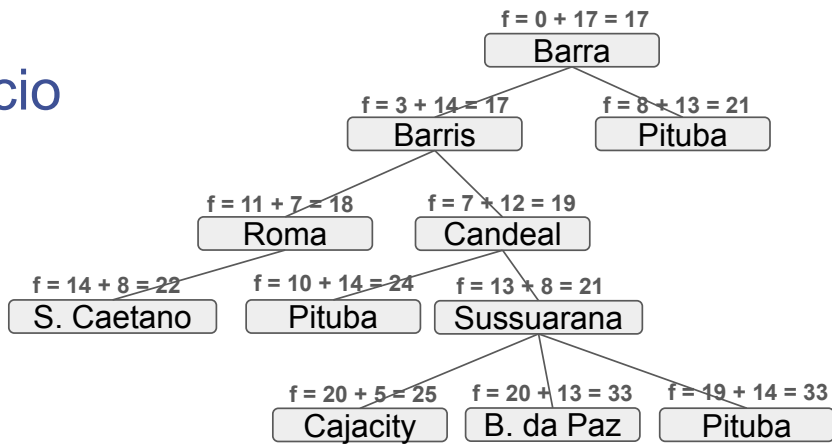
Exercício



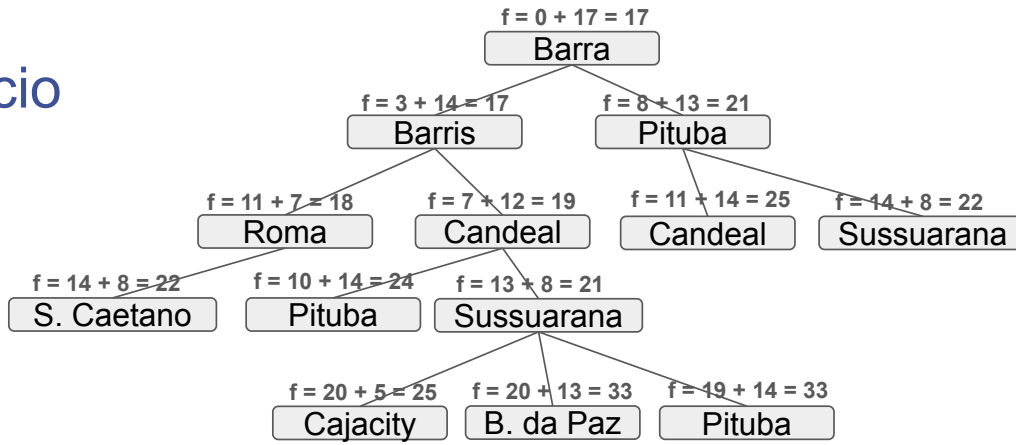
Empate! Como resolver?

- Ordem de chegada na Estrutura de dados (Pituba)
- Valor de $g(n)$
- **Valor de $h(n)$**
- Escolha aleatória

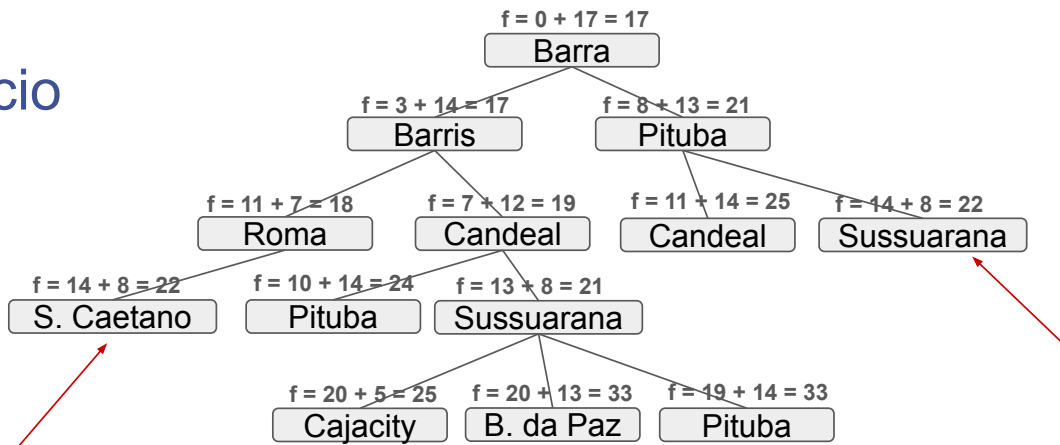
Exercício



Exercício



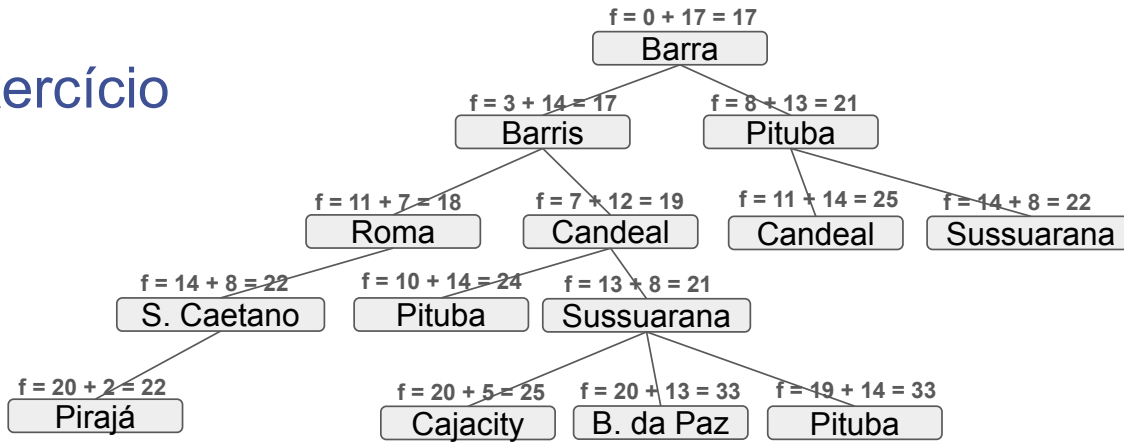
Exercício



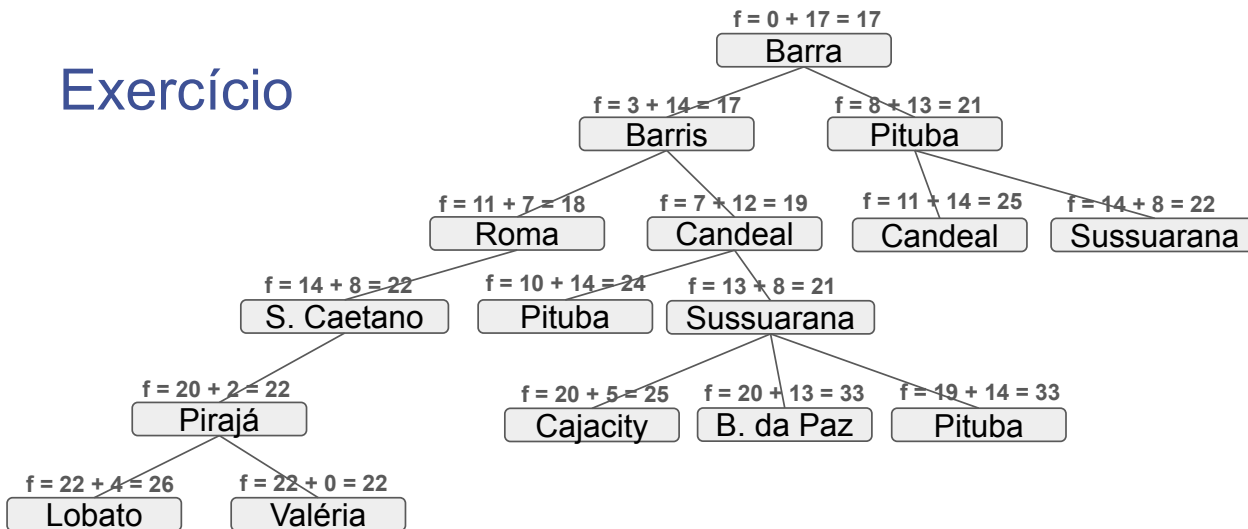
Empate! Como resolver?

- Nesse caso, Sussuarana não é escolhido, pois a escolha anterior tinha um $g(n)$ menor!

Exercício

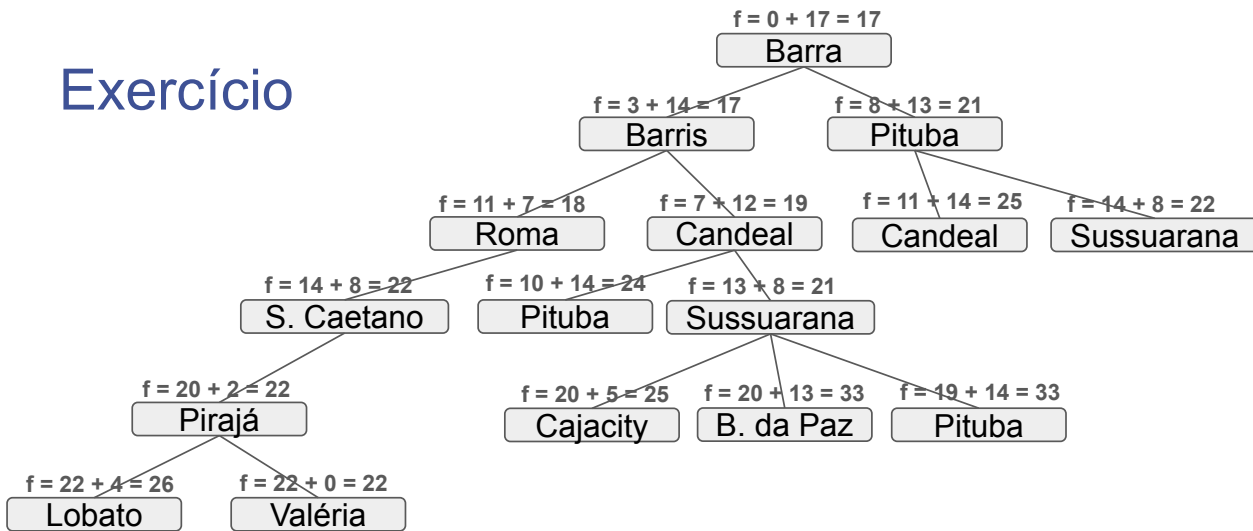


Exercício



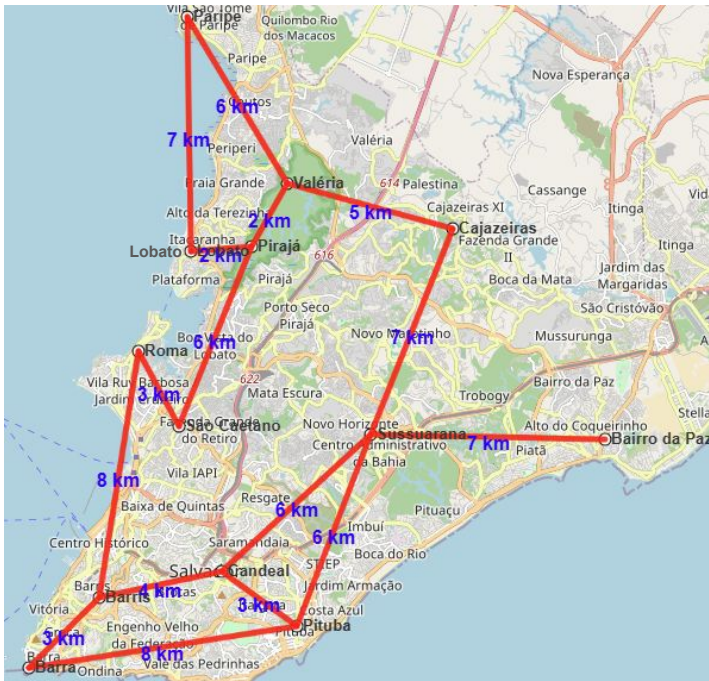


Exercício



Caminho: Barra → Barris → Roma → S. Caetano → Pirajá → Valéria
 $3\text{km} + 8\text{km} + 3\text{km} + 6\text{km} + 2\text{km} = 22\text{km}$

Exercício



Bairro	h(n)
Barra	17
Pituba	13
Barris	14
Candeal	12
Roma	7
São Caetano	8
Sussuarana	8
Cajazeiras	5
Bairro da Paz	13
Pirajá	2
Lobato	4
Valéria	0
Paripe	6



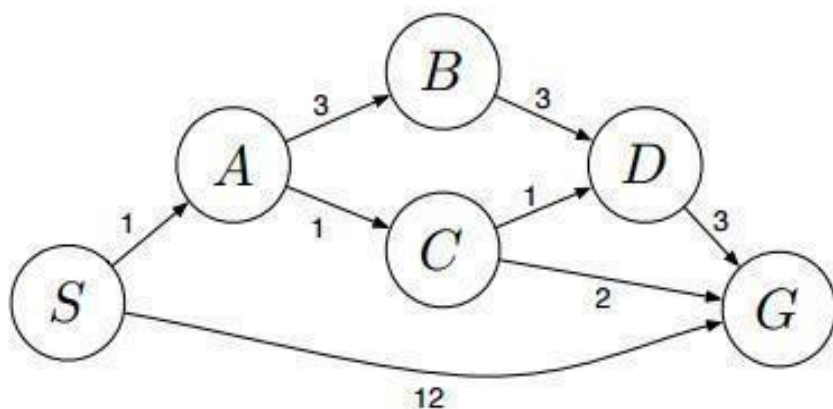
Algoritmo A*

- Admissível
 - Uma heurística admissível nunca superestima o custo de atingir o objetivo
 - $h(n)$: custo estimado saindo do estado n até a meta
 - $h^*(n)$: custo real saindo do estado n até a meta
 - Se $h(n) \leq h^*(n)$, então h é admissível



Exemplo

- Considere o espaço de estados abaixo e duas heurísticas possíveis, $h1$ e $h2$, na tabela ao lado.



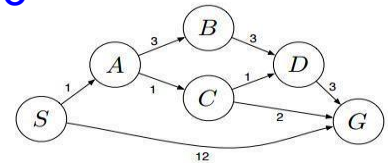
<i>Estado</i>	<i>h1</i>	<i>h2</i>
S	5	4
A	3	2
B	6	6
C	2	1
D	3	3
G	0	0





Exemplo

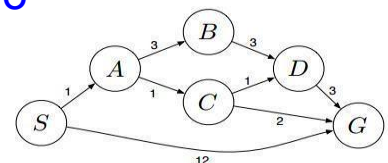
- Considere o espaço de estados abaixo e duas heurísticas possíveis, $h1$ e $h2$, na tabela ao lado.
 - $h1$ é admissível?
 - $h(n)$: estimativa de custo do nó n até o objetivo
 - $h^*(n)$: custo real do nó n até a meta
 - Se $h(n) \leq h^*(n)$, então h é admissível.



Estado	$h1$	$h2$
S	5	4
A	3	2
B	6	6
C	2	1
D	3	3
G	0	0

Exemplo

- Considere o espaço de estados abaixo e duas heurísticas possíveis, $h1$ e $h2$, na tabela ao lado.
 - $h1$ é admissível?
 - $h(n)$: estimativa de custo do nó n até o objetivo
 - $h^*(n)$: custo real do nó n até a meta
 - Se $h(n) \leq h^*(n)$, então h é admissível.



Estado	$h1$	$h2$
S	5	4
A	3	2
B	6	6
C	2	1
D	3	3
G	0	0

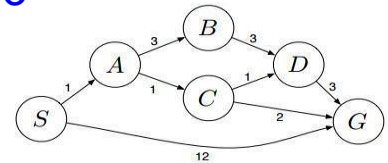
Re: Não, pois $h1(S) = 5$, mas $S \rightarrow A \rightarrow C \rightarrow G = 4$





Exemplo

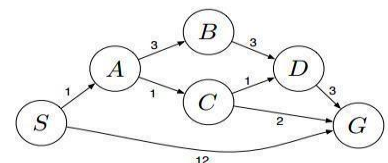
- Considere o espaço de estados abaixo e duas heurísticas possíveis, h_1 e h_2 , na tabela ao lado.
 - h_2 é admissível?
 - $h(n)$: estimativa de custo do nó n até o objetivo
 - $h^*(n)$: custo real do nó n até a meta
 - Se $h(n) \leq h^*(n)$, então h é admissível.



Estado	h_1	h_2
S	5	4
A	3	2
B	6	6
C	2	1
D	3	3
G	0	0

Exemplo

- Considere o espaço de estados abaixo e duas heurísticas possíveis, h_1 e h_2 , na tabela ao lado.
 - h_2 é admissível?
 - $h(n)$: estimativa de custo do nó n até o objetivo
 - $h^*(n)$: custo real do nó n até a meta
 - Se $h(n) \leq h^*(n)$, então h é admissível.



Estado	h_1	h_2
S	5	4
A	3	2
B	6	6
C	2	1
D	3	3
G	0	0

Re: Sim, pois

$h_2(S) = 4$, e $S \rightarrow A \rightarrow C \rightarrow G = 4$

$h_2(A) = 2$, e $A \rightarrow C \rightarrow G = 3$

$h_2(C) = 1$, e $C \rightarrow G = 2$

$h_2(D) = 3$, e $D \rightarrow G = 3$

$h_2(G) = 0$, $G \rightarrow G = 0$



Algoritmo A*

- Consistência (fórmula geral da desigualdade triangular)

$$h(n) \leq c(n, a, n') + h(n')$$



Algoritmo A*

- Consistência (fórmula geral da desigualdade triangular)

$$h(n) \leq c(n, a, n') + h(n')$$

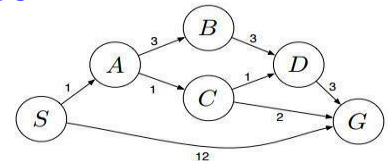
$$h(n) - h(n') \leq c(n, a, n')$$





Exemplo

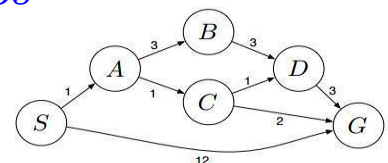
- Considere o espaço de estados abaixo e duas heurísticas possíveis, h1 e h2, na tabela ao lado.
 - h1 é consistente?
 - $h(n) - h(n') \leq \text{custo real de } n \text{ até } n', \text{ para todos os nós e caminhos}$



Estado	h1	h2
S	5	4
A	3	2
B	6	6
C	2	1
D	3	3
G	0	0

Exemplo

- Considere o espaço de estados abaixo e duas heurísticas possíveis, h1 e h2, na tabela ao lado.
 - h1 é consistente?
 - $h(n) - h(n') \leq \text{custo real de } n \text{ até } n', \text{ para todos os nós e caminhos}$



Re: Não, pois $h1(S) - h1(A) \leq \text{caminho } (S \rightarrow A)$
 $5 - 3 (\text{not}) \leq 1$

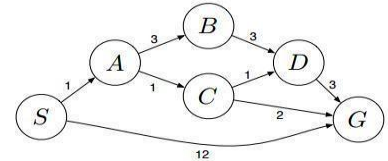
Estado	h1	h2
S	5	4
A	3	2
B	6	6
C	2	1
D	3	3
G	0	0





Exemplo

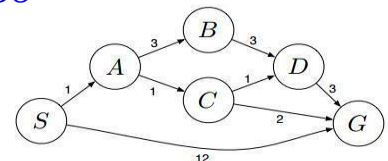
- Considere o espaço de estados abaixo e duas heurísticas possíveis, h1 e h2, na tabela ao lado.
 - h2 é consistente?
 - $h(n) - h(n') \leq \text{custo real de } n \text{ até } n', \text{ para todos os nós e caminhos}$



Estado	h1	h2
S	5	4
A	3	2
B	6	6
C	2	1
D	3	3
G	0	0

Exemplo

- Considere o espaço de estados abaixo e duas heurísticas possíveis, h1 e h2, na tabela ao lado.
 - h2 é consistente?
 - $h(n) - h(n') \leq \text{custo real de } n \text{ até } n', \text{ para todos os nós e caminhos}$



Re: Não, pois $h1(S) - h1(A) \leq \text{caminho } (S \rightarrow A)$
 $4 - 2 (\text{not}) \leq 1$

Estado	h1	h2
S	5	4
A	3	2
B	6	6
C	2	1
D	3	3
G	0	0





Algoritmo A*

- Otimalidade

$$\begin{aligned}f(n) &\leq f(n') \\g(n) + h(n) &\leq g(n') + h(n') \\g(n) + h(n) &\leq g(n) + c(n, a, n') + h(n')\end{aligned}$$

- Assim, dizemos que $h(n)$ é consistente visto que os valores de $f(n)$ ao longo do caminho são não decrescentes.

Algoritmo A*

- Uma heurística consistente pode ser admissível?

$$\begin{aligned}h(n) &\leq c(n, a, n') + h(n') \\h(n) &\leq c(n, a, n') + h(n') \leq c(n, a, n') + h^*(n') \\h(n) &\leq c(n, a, n') + h(n') \leq c(n, a, n') + h^*(n') = h^*(n)\end{aligned}$$



Referência Bibliográfica

- RUSSELL, S.; NORVIG, P. Inteligência Artificial. Editora Campus, 2013, Cap. 03.

